



angular

دليل تعليمي مختصر و منظم

Amine Mohamed

Telegram: @codeDarija



جدول المحتويات

1. مقدمة إلى Angular و TypeScript — ما هو Angular ولماذا نستخدمه؟
2. مقدمة إلى Angular و TypeScript — إعداد بيئة التطوير (Node.js, npm, Angular CLI)
3. مقدمة إلى Angular و TypeScript — إنشاء أول مشروع Angular باستخدام CLI
4. مقدمة إلى Angular و TypeScript — نظرة عامة على بنية مشروع Angular
5. مقدمة إلى Angular و TypeScript — مقدمة إلى TypeScript: الأنواع، الواجهات، الأصناف
6. المكونات (Components) — مفهوم المكونات في Angular
7. المكونات (Components) — إنشاء المكونات باستخدام CLI
8. المكونات (Components) — بنية المكون: ملفات HTML, CSS, TS
9. المكونات (Components) — خاصية `@Component` ومحددات المكون
10. المكونات (Components) — تداخل المكونات (Component Nesting)
11. القوالب وربط البيانات (Templates & Data Binding) — بناء قوالب HTML في Angular
12. القوالب وربط البيانات (Templates & Data Binding) — ربط البيانات أحادي الاتجاه (One-Way Data Binding): الاستيفاء وربط الخصائص

13. القوالب وربط البيانات (Templates & Data Binding) — ربط البيانات أحادي الاتجاه (One-Way Data Binding): ربط الأحداث
14. القوالب وربط البيانات (Templates & Data Binding) — ربط البيانات ثنائي الاتجاه (Two-Way Data Binding) باستخدام ``ngModel``
15. التوجيهات (Directives) — مقدمة إلى التوجيهات وأنواعها
16. التوجيهات (Directives) — التوجيهات الهيكلية (Structural Directives): ``ngIf``, ``*ngFor``
17. التوجيهات (Directives) — التوجيهات الهيكلية (Structural Directives): ``*ngSwitch``
18. التوجيهات (Directives) — التوجيهات السماتية (Attribute Directives): ``[ngStyle]``, ``[ngClass]``
19. التوجيهات (Directives) — إنشاء توجيهات سماتية مخصصة (Custom Attribute Directives)
20. الخدمات وحقن الاعتمادية (Services & Dependency Injection) — مفهوم الخدمات ولماذا نستخدمها
21. الخدمات وحقن الاعتمادية (Services & Dependency Injection) — إنشاء خدمة باستخدام CLI
22. الخدمات وحقن الاعتمادية (Services & Dependency Injection) — حقن الاعتمادية (Dependency Injection): مفهومها وكيفية عملها
23. الخدمات وحقن الاعتمادية (Services & Dependency Injection) — استخدام الخدمات في المكونات
24. الخدمات وحقن الاعتمادية (Services & Dependency Injection) — فهم الأنواع المختلفة لمزودي الخدمات (Providers)
25. التوجيه (Routing) — مفهوم التوجيه في تطبيقات الصفحة الواحدة (SPA)

26. التوجيه (Routing) — إعداد وحدة التوجيه (App Routing Module)
27. التوجيه (Routing) — تعريف المسارات (Routes) والتوجيه الأساسي
28. التوجيه (Routing) — الروابط الموجهة (Router Links) و router-`outlet`
29. التوجيه (Routing) — تمرير المعلومات في المسارات (Route Parameters)
30. التوجيه (Routing) — الحراس (Route Guards) للحماية (مقدمة بسيطة)
31. النماذج (Forms) — مقدمة إلى نماذج Angular
32. النماذج (Forms) — النماذج القائمة على القوالب (Template-Driven Forms): الأساسيات و`FormsModule`
33. النماذج (Forms) — النماذج القائمة على القوالب: التحقق من الصحة (Validation)
34. النماذج (Forms) — النماذج التفاعلية (Reactive Forms): `FormControl`, `FormGroup`
35. النماذج (Forms) — النماذج التفاعلية: `FormArray`
36. النماذج (Forms) — النماذج التفاعلية: التحقق من الصحة المخصص (Custom Validation)
37. عميل (HTTP Client) HTTP — مقدمة لطلب البيانات من الخوادم
38. عميل (HTTP Client) HTTP — وحدة `HttpClientModule` وإجراء طلبات GET
39. عميل (HTTP Client) HTTP — إجراء طلبات POST, PUT, DELETE
40. عميل (HTTP Client) HTTP — التعامل مع الاستجابات والأخطاء (Error Handling)

41. عميل HTTP (HTTP Client) — المرتقبات (Observables) في سياق HTTP

42. المرتقبات (Observables) RxJSg — مقدمة إلى RxJSg ReactiveX

43. المرتقبات (Observables) RxJSg — مفهوم المرتقبات (Observables) والمراقبين (Observers)

44. المرتقبات (Observables) RxJSg — الاشتراك في المرتقبات (Subscribing) وإلغاء الاشتراك

45. المرتقبات (Observables) RxJSg — العوامل الشائعة (Operators): ``map``, ``filter``, ``debounceTime``

46. المرتقبات (Observables) RxJSg — العوامل الشائعة: ``switchMap``, ``tap``

47. الأنابيب (Pipes) — مفهوم الأنابيب ودورها في تحويل البيانات

48. الأنابيب (Pipes) — الأنابيب المدمجة (Built-in Pipes): ``DatePipe``, ``CurrencyPipe``

49. الأنابيب (Pipes) — الأنابيب المدمجة: ``JsonPipe``, ``SlicePipe``, ``AsyncPipe``

50. الأنابيب (Pipes) — استخدام الأنابيب في القوالب

51. الأنابيب (Pipes) — إنشاء أنابيب مخصصة (Custom Pipes)

52. إدارة الحالة الأساسية (Basic State Management) — مقدمة لمفهوم إدارة الحالة في تطبيقات الويب

53. إدارة الحالة الأساسية (Basic State Management) — مشاركة البيانات بين المكونات باستخدام الخدمات و ``BehaviorSubject``

54. إدارة الحالة الأساسية (Basic State Management) — استخدام
التخزين المحلي (LocalStorage) للحالة البسيطة

مقدمة إلى Angular و TypeScript — ما هو Angular ولماذا نستخدمه؟

html

ما هو Angular ولماذا نستخدمه؟

Angular هو إطار عمل (Framework) شامل لبناء تطبيقات الويب المعقدة والديناميكية، خاصةً تطبيقات الصفحة الواحدة (Single Page Applications أو SPAs). تم تطويره وصيانته بواسطة جوجل، ويُستخدم لبناء واجهات المستخدم التفاعلية التي تعمل بكفاءة عالية على مختلف المتصفحات.

Angular كإطار عمل شامل

على عكس المكتبات (مثل React)، يوفر Angular مجموعة متكاملة من الأدوات والحلول الجاهزة لإدارة التوجيه (Routing)، وإدارة الحالة، والتواصل مع الخوادم. هذا الهيكل المنظم يفرض طريقة معينة لكتابة الكود، مما يجعل المشاريع الكبيرة أسهل في الصيانة والتطوير الجماعي. يعتمد Angular بشكل أساسي على لغة TypeScript، وهي نسخة محسنة من JavaScript تضيف ميزات كتابة الأنواع (Static Typing) لزيادة موثوقية الكود.

مثال على هيكل المكون (Component)

في Angular، تُبنى التطبيقات من خلال "مكونات" (Components)، يمثل كل منها جزءًا من واجهة المستخدم المستخدم (مثل شريط التنقل أو بطاقة المنتج). هذا مثال بسيط لمكون TypeScript:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello-world',
  template: 'مرحباً بالعالم!'
})
export class HelloWorldComponent {
  // يمكن إضافة منطق المكون هنا
}
```


شرح الكود

يوضح هذا المثال تعريف مكون بسيط. تُستخدم دالة `@Component` لتحديد خصائص المكون: `selector` هو الوسم الخاص (tag name) الذي نستخدمه في HTML لاستدعاء هذا المكون، و `template` هو المحتوى الذي سيتم عرضه. الكلاس `HelloWorldComponent` هو المكان الذي نكتب فيه المنطق الخاص بهذا المكون باستخدام TypeScript.

- **إطار عمل شامل:** يوفر Angular أدوات متكاملة لبناء تطبيقات الويب المعقدة من البداية.
- **يعتمد على TypeScript:** استخدام TypeScript يقلل من الأخطاء ويزيد من سهولة صيانة الكود.
- **تطبيقات SPA:** مثالي لبناء تطبيقات الصفحة الواحدة التي توفر تجربة مستخدم سريعة وديناميكية.

...

مقدمة إلى Angular و TypeScript — إعداد بيئة التطوير (Node.js, npm, Angular CLI)

إعداد بيئة التطوير (Node.js, npm, Angular CLI)

لإنشاء تطبيق Angular، نحتاج أولاً إلى إعداد بيئة التطوير على جهاز الكمبيوتر الخاص بك. يعتمد Angular بشكل كبير على أدوات نظام JavaScript البيئي. في هذا الدرس، سنتعرف على الأدوات الأساسية التي يجب تثبيتها: Node.js، و npm، و Angular CLI.

الأدوات الأساسية لبيئة Angular

Angular هو إطار عمل يعتمد على TypeScript، ولكنه يعمل ضمن بيئة Node.js. Node.js هو بيئة تشغيل JavaScript تسمح بتشغيل أدوات البناء خارج المتصفح. **npm** (مدير حزم Node) هو الأداة القياسية لإدارة مكتبات ومكونات المشروع. أما **Angular CLI** (واجهة سطر الأوامر) فهي الأداة الأقوى والأكثر أهمية لإنشاء المشاريع، وإنشاء المكونات، وتشغيل التطبيق المحلي.

مثال على إعداد المشروع

```
npm install -g @angular/cli
ng new my-first-app
cd my-first-app
ng serve --open
```

شرح الكود

1. `npm install -g @angular/cli`: يقوم هذا الأمر بتثبيت Angular CLI عالميًا على جهازك. هذا يسمح لك باستخدام الأمر `ng` من أي مكان.
2. `ng new my-first-app`: ينشئ هذا الأمر مشروع Angular جديدًا بالكامل باسم "my-first-app" ويتضمن جميع الملفات والإعدادات اللازمة.

3. `ng serve --open`: يقوم هذا الأمر بتشغيل خادم محلي، ويفتح التطبيق تلقائيًا في متصفحك، مما يسمح لك برؤية التغييرات فور حفظها.

- **Node.js**: يوفر بيئة التشغيل الأساسية لتشغيل أدوات Angular.
- **npm**: يُستخدم لإدارة حزم ومكتبات المشروع، مثل تثبيت Angular والمكتبات الإضافية.
- **Angular CLI**: هي الأداة الرئيسية التي تُسهل إنشاء المشاريع وتشغيلها وتطويرها بكفاءة.

مقدمة إلى Angular و TypeScript — إنشاء أول مشروع Angular باستخدام CLI

html™

إنشاء أول مشروع Angular باستخدام CLI

في هذا الدرس، سنتعلم كيفية استخدام Angular CLI (واجهة سطر الأوامر) لإنشاء وتجهيز أول مشروع Angular. Angular CLI هي الأداة الرسمية التي تبسط عملية تطوير تطبيقات Angular بشكل كبير، حيث تقوم بإنشاء جميع الملفات الضرورية وإعداد بيئة العمل تلقائيًا.

تجهيز البيئة وإنشاء المشروع

لإنشاء مشروع Angular، يجب أولاً تثبيت Angular CLI على جهازك. بعد التثبيت، نستخدم الأمر `ng new` لإنشاء هيكل التطبيق الأساسي، والذي يتضمن ملفات TypeScript و HTML و CSS.

مثال: إنشاء مشروع جديد وتشغيله

```
npm install -g @angular/cli
ng new my-first-app
cd my-first-app
ng serve --open
```

شرح الكود

يبدأ الكود بتثبيت Angular CLI عالميًا (`npm install -g @angular/cli`). بعد ذلك، يقوم الأمر `ng new my-first-app` بإنشاء مجلد جديد باسم `my-first-app` يحتوي على كافة ملفات المشروع الأساسية. ثم ننتقل إلى المجلد باستخدام `cd`. وأخيرًا، يقوم الأمر `ng serve --open` بتشغيل خادم التطوير المحلي وفتح التطبيق في المتصفح تلقائيًا لمشاهدة النتيجة.

ملخص الدرس

- Angular CLI هي الأداة القياسية لإدارة مشاريع Angular وتسهيل مهام التطوير.
- يستخدم الأمر `ng new` لإنشاء هيكل مشروع Angular جديد جاهز للتشغيل.
- يستخدم الأمر `ng serve` لتشغيل التطبيق على خادم تطوير محلي وعرضه في المتصفح.

مقدمة إلى Angular و TypeScript — نظرة عامة

على بنية مشروع Angular

نظرة عامة على بنية مشروع Angular

عند إنشاء مشروع Angular جديد، يتم إنشاء بنية مجلدات قياسية. فهم هذه البنية ضروري لتحديد مكان كتابة الأكواد الخاصة بك وكيفية عمل التطبيق. سنتعرف في هذا الدرس على أهم المجلدات والملفات التي يجب أن تعرفها كمبتدئ.

المكونات الرئيسية للبنية

يتم تنظيم مشاريع Angular حول مجلد `src`، وهو المكان الذي يوجد فيه معظم كود التطبيق الخاص بك. داخل `src`، ستجد مجلد `app` الذي يحتوي على المكونات (components) الأساسية للتطبيق.

بنية المجلدات الأساسية

```
angular-project/  
├─ node_modules/  
├─ src/  
│   ├─ app/  
│   │   ├─ app.component.html  
│   │   ├─ app.component.css  
│   │   ├─ app.component.ts  
│   │   └─ app.module.ts  
│   ├─ assets/  
│   ├─ environments/  
│   ├─ index.html  
│   └─ main.ts  
├─ angular.json  
└─ package.json
```

شرح الكود

البنية الموضحة هي الهيكل الأساسي لأي مشروع Angular. أهم المكونات هي:

- **مجلد src**: يحتوي على جميع ملفات مصدر التطبيق (Source code). هذا هو المكان الذي ستقضي فيه معظم وقتك.
- **مجلد app**: يضم المكونات (components) الخاصة بتطبيقك، وهي اللبنة الأساسية لمواجهة المستخدم. كل مكون يتكون عادةً من ملفات HTML (ال قالب)، CSS (التنسيق)، و TypeScript (المنطق).
- **ملف index.html**: نقطة الدخول الرئيسية للتطبيق. هو الصفحة الوحيدة التي يتم تحميلها في المتصفح، ويتم حقن تطبيق Angular داخلها.
- **ملف main.ts**: ملف بدء التشغيل (Bootstrapping) للتطبيق، يخبر Angular أي مكون يجب أن يبدأ به التطبيق.
- **ملف angular.json**: ملف إعدادات (Configuration) مهم لإدارة إعدادات المشروع بالكامل، مثل إعدادات البناء (build) والبيئات المختلفة.
- **ملف package.json**: يحتوي على قائمة بجميع مكتبات Angular والتبعيات الأخرى التي يحتاجها المشروع.

ملخص البنية

- **المجلد src** هو قلب المشروع ويحتوي على الكود الفعلي لتطبيقك.
- **ملف index.html** هو نقطة البداية لتطبيق الصفحة الواحدة (SPA) في المتصفح.
- يتم تعريف المكونات الرئيسية للتطبيق داخل **مجلد app**.

مقدمة إلى Angular و TypeScript — مقدمة إلى TypeScript: الأنواع، الواجهات، الأصناف

html

مقدمة إلى TypeScript: الأنواع، الواجهات، الأصناف

Angular يعتمد بشكل أساسي على TypeScript. تهدف TypeScript إلى إضافة ميزة الكتابة الثابتة (Static Typing) إلى JavaScript. هذا يعني أنه يمكنك تحديد نوع البيانات للمتغيرات (مثل رقم أو نص)، مما يساعد في اكتشاف الأخطاء مبكراً أثناء كتابة الكود بدلاً من وقت تشغيل البرنامج. هذا الدرس يوضح أهم ثلاثة مفاهيم أساسية في TypeScript: الأنواع، الواجهات، والأصناف.

مفهوم الأنواع (Types)

تسمح لك TypeScript بتحديد نوع البيانات التي يمكن أن يحتويها المتغير، مثل `string` للنصوص، `number` للأرقام، `boolean` للقيم المنطقية، و `Array` للمصفوفات. هذا يساعد في الحفاظ على تناسق البيانات ويمنع الأخطاء غير المتوقعة.

مفهوم الواجهات (Interfaces)

الواجهة هي عبارة عن عقد (Contract) يحدد الشكل الذي يجب أن يكون عليه الكائن (Object). إذا كان لديك كائن يمثل "مستخدم" مثلاً، يمكنك استخدام واجهة لتحديد أن هذا الكائن يجب أن يحتوي على خصائص معينة مثل `id` و `name`.

مفهوم الأصناف (Classes)

الأصناف (Classes) هي قوالب لإنشاء الكائنات (Objects). تجمع الأصناف بين البيانات (الخصائص) والوظائف (الأساليب) التي تعمل على هذه البيانات. تُستخدم الأصناف على نطاق واسع في Angular لإنشاء المكونات (Components) والخدمات (Services).

مثال تطبيقي (الواجهات والأصناف)

```
interface UserInterface {
  id: number;
  name: string;
  email: string;
}

class User implements UserInterface {
  constructor(public id: number, public name: string, public email: string) {}

  greetUser(): string {
    return `Hello, my name is ${this.name}`;
  }
}

let user1 = new User(1, "Ahmed", "ahmed@example.com");
console.log(user1.greetUser());
```

شرح الكود

يبدأ الكود بتعريف واجهة (`UserInterface`) تحدد هيكل كائن المستخدم. ثم ننشئ صنف (`User`) يطبق هذه الواجهة. الصنف `User` يحتوي على خصائص (مثل `id` و `name`) ودالة (`greetUser`) تعمل على هذه الخصائص. عند إنشاء كائن جديد (`user1`)، فإن TypeScript تضمن أنه يلتزم بالهيكل المحدد في الواجهة.

النقاط الرئيسية

- **الأنواع (Types):** تساعد في تحديد أنواع البيانات لتقليل الأخطاء وزيادة وضوح الكود.
- **الواجهات (Interfaces):** تحدد الهيكل المتوقع للكائنات وتساعد في تنظيم البيانات.
- **الأصناف (Classes):** تُستخدم لإنشاء قوالب للكائنات وتجميع الخصائص والوظائف ذات الصلة.

المكونات (Components) — مفهوم المكونات في Angular

مفهوم المكونات في Angular

تعتبر المكونات (Components) حجر الزاوية في بناء تطبيقات Angular. إنها الطريقة التي يتم بها تقسيم واجهة المستخدم (UI) إلى وحدات مستقلة قابلة لإعادة الاستخدام. فهم المكونات هو الخطوة الأولى لإتقان Angular.

ما هو المكون (Component)؟

المكون هو عبارة عن جزء مستقل بذاته من واجهة المستخدم. يقوم المكون بدمج المنطق (Logic) الخاص به مع العرض (Presentation) الخاص به. كل مكون يتكون بشكل أساسي من: **ملف TypeScript** (يحتوي على المنطق والبيانات)، و**ملف HTML** (للعرض)، و**ملف CSS** (للتنسيق).

مثال على تعريف المكون

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-greeting',
  template: `
    <h1>مرحباً بك في تطبيق Angular!</h1>
    <p>هذا المكون يعرض رسالة ترحيب بسيطة</p>
  `,
  styles: [`
    h1 { color: #3f51b5; }
    p { font-size: 16px; }
  `]
})
export class GreetingComponent {
  // يمكن إضافة منطق المكون هنا
}
```

شرح الكود

الكود أعلاه يعرف مكوناً جديداً باستخدام المزخرف `@Component`. هذا المزخرف يخبر Angular بأن هذا الكلاس (Class) هو مكون. نحن نحدد الخصائص الأساسية للمكون: `selector` وهو الاسم الذي سيستخدم لاستدعاء المكون في HTML (مثل `<app-greeting></app-greeting>`)، و `template` وهو قالب HTML المرئي للمكون، و `styles` لتنسيق CSS الخاص بالمكون.

- **بناء التطبيق:** تطبيقات Angular هي شجرة من المكونات المتداخلة، حيث يحتوي المكون الأب على مكونات أبناء.
- **التغليف (Encapsulation):** كل مكون يغلف منطق عرضه وتصميمه بشكل مستقل، مما يجعله سهل الصيانة والفصل.
- **إعادة الاستخدام:** يمكن استدعاء نفس المكون عدة مرات في أجزاء مختلفة من التطبيق لضمان الاتساق وتقليل تكرار الكود.

المكونات (Components) — إنشاء المكونات باستخدام CLI

html

إنشاء المكونات باستخدام CLI

المكونات (Components) هي اللبنات الأساسية لتطبيقات Angular، حيث تتحكم كل واحدة منها بجزء معين من واجهة المستخدم. لإنشاء مكون جديد، بدلاً من إنشاء ملفات TypeScript وHTML وCSS يدوياً، نستخدم أداة سطر الأوامر (CLI) الخاصة بـ Angular. هذا يوفر الوقت ويضمن التكوين الصحيح للمشروع.

الأمر الأساسي لتوليد المكونات

يستخدم Angular CLI الأمر `ng generate component` (أو اختصاراً `ng g c`) لإنشاء جميع الملفات الضرورية للمكون الجديد تلقائياً. عند تشغيل هذا الأمر، يقوم CLI بما يلي:

- إنشاء ملف TypeScript (.ts) للمنطق.
- إنشاء ملف HTML (.html) لل قالب.
- إنشاء ملف CSS (.css) للتصميم.
- إنشاء ملف اختبار (.spec.ts).
- تحديث ملف `app.module.ts` لتسجيل المكون الجديد تلقائياً.

مثال على الأمر

```
ng generate component user-profile
```

شرح الكود

في هذا المثال، نستخدم الأمر `ng generate component` لإنشاء مكون جديد باسم `user-profile`. سيقوم CLI بإنشاء مجلد جديد باسم `user-profile` يحتوي على الملفات الأربعة الخاصة بالمكون، كما سيقوم بتسجيل المكون في الوحدة (Module) الرئيسية للتطبيق (عادةً `app.module.ts`).

ملخص الفكرة

- **أداة CLI:** استخدم `<ng g c <name>` لإنشاء مكونات جديدة بسرعة وكفاءة.
- **التوليد التلقائي:** تقوم الأداة بتوليد أربعة ملفات أساسية للمكون في مجلد خاص به.
- **التسجيل التلقائي:** يقوم CLI بتحديث ملف الوحدة (Module) الرئيسي تلقائياً، مما يلغي الحاجة للتسجيل اليدوي.

المكونات (Components) — بنية المكون: ملفات HTML, CSS, TS

بنية المكون: ملفات HTML, CSS, TS

في إطار عمل Angular، يعد المكون (Component) الوحدة الأساسية لبناء واجهة المستخدم. يتم تصميم كل مكون عادةً ليجمع بين المنطق (Logic)، والهيكل (Structure)، والمظهر (Appearance) في مكان واحد منظم، مما يجعل عملية التطوير أكثر كفاءة ومرونة.

الفصل بين المهام (Separation of Concerns)

يتم تحقيق بنية المكون في Angular من خلال ثلاثة ملفات رئيسية لكل منها وظيفة محددة، مما يتبع مبدأ الفصل بين المهام:

- **ملف TypeScript (TS):** يمثل المنطق البرمجي للمكون. يحتوي هذا الملف على الخصائص (Properties) والدوال (Methods) التي تتحكم في البيانات وسلوك المكون.
- **ملف HTML:** يمثل القالب (Template) الخاص بالمكون. يحدد هذا الملف الهيكل البصري للمكون وكيفية عرض البيانات للمستخدم.
- **ملف CSS:** يمثل الأنماط (Styles) الخاصة بالمكون. يحدد مظهر المكون من حيث الألوان، الخطوط، والتنسيق، وهو خاص بهذا المكون فقط.

مثال على ربط ملفات المكون

يتم ربط هذه الملفات معًا باستخدام مزين `@Component` في ملف TypeScript الرئيسي للمكون:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-profile',
  templateUrl: './user-profile.component.html',
  styleUrls: ['./user-profile.component.css']
})
export class UserProfileComponent {
```

```
userName: string = 'Ahmed Mohamed';  
}
```

شرح الكود

في هذا المثال، ملف `user-profile.component.ts` هو العقل المدبر للمكون. يُستخدم المزين `Component@` لربط ملف المنطق (TS) بملف الهيكلية (HTML) `templateUrl` وملف الأنماط (CSS) `styleUrls`. هذا الربط يضمن أن البيانات والمنطق الموجود في ملف TS سيتم تطبيقهما على القالب (HTML) وتنسيقهما بواسطة ملف CSS.

ملخص بنية المكون

- **المنطق:** يدار بواسطة ملف TypeScript.
- **الهيكلية:** يتم تحديدها بواسطة ملف HTML.
- **المظهر:** يتم تنسيقه بواسطة ملف CSS.

المكونات (Components) — خاصية `@Component` ومحددات المكون

html

خاصية @Component ومحددات المكون

في إطار تعلم Angular، المكونات (Components) هي اللبنات الأساسية التي تُشكل واجهة المستخدم. يتم تعريف المكون باستخدام خاصية تُسمى `@Component`، وهي عبارة عن "ديكوريتور" (Decorator) يخبر Angular بأن فئة (Class) معينة هي في الواقع مكون، ويحدد خصائصه الأساسية.

محدد المكون (Selector)

محدد المكون هو الاسم الذي يُستخدم في ملفات HTML كـ "وسم مخصص" (Custom Tag) لإدراج المكون في الصفحة. هو الرابط الأساسي الذي يربط فئة المكون (TypeScript Class) بملف القالب الخاص به (HTML Template) في واجهة المستخدم. تحدد هذه الخاصية أين سيتم عرض المكون على الشاشة.

مثال على الكود

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello',
  templateUrl: './hello.component.html',
  styleUrls: ['./hello.component.css']
})
export class HelloComponent {
  title = 'مرحباً بالعالم';
}
```

شرح الكود

في هذا المثال، قمنا بتعريف المكون `HelloComponent`. الخاصية `@Component` هي التي تجعله مكونًا في Angular. تحتوي هذه الخاصية على:

- `'selector: 'app-hello'`: هذا هو المحدد. سيتم استخدام الوسم `<app-hello></app-hello>` في ملف HTML الرئيسي لعرض محتوى هذا المكون.
- `templateUrl`: يشير إلى مسار ملف HTML الذي يحتوي على تصميم المكون (القالب).
- `styleUrls`: يشير إلى مسار ملف CSS الخاص بتنسيق المكون.

ملخص الدرس

- خاصية `@Component` هي طريقة لتعريف الفئات كمكونات في Angular.
- `selector` هو الاسم الفريد للمكون الذي يُستخدم لإدراجه في ملفات HTML.
- يجب أن يحتوي كل مكون على محدد `selector` وقالب `template` (سواء كان `templateUrl` أو `template`).

المكونات (Components) — تداخل المكونات (Component Nesting)

تداخل المكونات (Component Nesting)

في Angular، تُعد المكونات بمثابة لبنات البناء الأساسية. يتيح لك تداخل المكونات إنشاء تطبيقات معقدة عن طريق تجميع مكونات أصغر داخل مكونات أكبر، تماماً مثل تجميع أجزاء ليغو (Lego) لتشكيل بناء متكامل. هذا المفهوم ضروري لإنشاء واجهات مستخدم قابلة لإعادة الاستخدام ومنظمة.

كيف يعمل تداخل المكونات؟

يحدث التداخل عندما يستخدم المكون الأب (Parent Component) محدد المكون الابن (Child Component's selector) داخل قالب HTML الخاص به. عند عرض المكون الأب، يتم عرض محتوى المكون الابن تلقائياً في المكان الذي تم فيه وضع المحدد.

مثال على تداخل المكونات

لنفترض أن لدينا مكوناً بسيطاً للترحيب (`GreetingComponent`) ومكوناً رئيسياً للوحة التحكم (`DashboardComponent`). سنقوم بدمج مكون الترحيب داخل لوحة التحكم.

1. مكون الابن (`GreetingComponent`):

```
<!-- greeting.component.html -->
<div class="greeting-message">
  <h1>مرحباً بك في لوحة التحكم</h1>
</div>
```

2. مكون الأب (`DashboardComponent`):

```
<!-- dashboard.component.html -->
<div class="dashboard-container">
  <app-greeting></app-greeting> <!-- تضمين مكون الابن -->
```

```
<p>...هنا سيتم عرض باقي محتوى لوحة التحكم</p>  
</div>
```

شرح الكود

في المثال أعلاه، قمنا بتعريف مكون الترحيب (الابن) بمحتوى ثابت. ثم استخدمنا محدد هذا المكون (`<app-greeting>`) داخل قالب المكون الأب (`DashboardComponent`). عندما يتم عرض لوحة التحكم، سيعرض Angular تلقائياً محتوى الترحيب (`<h1>مرحباً...</h1>`) في المكان المحدد داخل قالب الأب.

- **إعادة الاستخدام:** يمكنك استخدام نفس المكون الابن (مثل `<app-greeting>`) في أي مكان آخر داخل التطبيق.
- **التنظيم:** يساعد التداخل في تقسيم الواجهة المعقدة إلى مكونات أصغر، مما يسهل إدارة الكود.
- **العلاقة الأبوية/الابن:** المكون الأب يحدد متى وأين سيتم عرض المكون الابن، مما يتيح بناء هياكل مرنة للتطبيق.

القوالب وربط البيانات (Templates & Data Binding)

— بناء قوالب HTML في Angular

بناء قوالب HTML في Angular

القالب (Template) هو واجهة المستخدم (User Interface) المرئية للمكون (Component) في Angular. بعبارة أخرى، هو ملف HTML يحدد الشكل الذي سيظهر به المكون على الشاشة. في هذا الدرس، نتعلم كيفية بناء هذا القالب الأساسي وكيفية ربطه بمنطق المكون.

مفهوم القالب (Template)

يتم استخدام HTML لوصف بنية القالب. يمكن تعريف القالب بإحدى طريقتين:

- **القالب الخارجي (External Template):** وهو الطريقة الأكثر شيوعًا، حيث يتم وضع HTML في ملف منفصل (عادةً `component.html`) ويتم الإشارة إليه في ملف TypeScript الخاص بالمكون باستخدام الخاصية `templateUrl`.
- **القالب المدمج (Inline Template):** يتم كتابة HTML مباشرة داخل ملف TypeScript باستخدام الخاصية `template`.

مثال: ربط قالب HTML ببيانات المكون

```
<!-- app.component.html -->
<div>
  <h1>مرحباً {{ title }}</h1>
  <p>هذا المكون يعمل بشكل جيد</p>
  <button (click)="changeTitle()">تغيير العنوان</button>
</div>
```

شرح الكود

يوضح المثال أعلاه قالباً بسيطاً يحتوي على عناصر HTML عادية. الميزة الأساسية هي استخدام **التنصيص (Interpolation)**: `{{ title }}`. هذه الصيغة تقوم بربط القيمة الموجودة في المتغير `title` داخل ملف TypeScript وعرضها في القالب. عند تغيير قيمة `title` في ملف TypeScript، يتم تحديث القالب تلقائياً.

- **HTML عادي:** يمكن استخدام أي عنصر HTML قياسي (مثل `<h1>` و `<p>`) في قوالب Angular.
- **التنصيص (Interpolation):** يتم استخدام الأقواس المزدوجة `{{ }}` لعرض بيانات المكون (القيم الموجودة في TypeScript) داخل القالب.
- **ربط الأحداث (Event Binding):** يتم استخدام الأقواس العادية `(click)` لربط الأحداث التي تحدث في القالب بوظائف (methods) موجودة في TypeScript.

القوالب وربط البيانات (Templates & Data Binding)

— ربط البيانات أحادي الاتجاه (One-Way Data Binding): الاستيفاء وربط الخاصيات

html

ربط البيانات أحادي الاتجاه: الاستيفاء وربط الخاصيات

في Angular، ربط البيانات أحادي الاتجاه (One-Way Data Binding) هو عملية نقل البيانات من ملف TypeScript (الكومبوننت) إلى قالب HTML (العرض) الخاص بالكومبوننت. هذا يعني أن التغييرات في الكومبوننت تنعكس على العرض، ولكن العكس ليس صحيحًا في هذا النوع من الربط. هناك طريقتان رئيسيتان لتحقيق ذلك: الاستيفاء (Interpolation) وربط الخاصيات (Property Binding).

مفهوم الاستيفاء وربط الخاصيات

1. الاستيفاء (Interpolation): يُستخدم لعرض قيمة متغير من الكومبوننت مباشرةً كنص داخل عنصر HTML. يتم تمثيله باستخدام الأقواس المعقوفة المزدوجة `{{ }}`. هذه هي الطريقة الأبسط لعرض البيانات.

2. ربط الخاصيات (Property Binding): يُستخدم لتعيين قيمة خاصية (Property) معينة لعنصر HTML (مثل `src` للصورة أو `disabled` للزر) بناءً على قيمة متغير في الكومبوننت. يتم تمثيله باستخدام الأقواس المربعة `[]`.

مثال على الكود

```
// app.component.ts
export class AppComponent {
  pageTitle: string = 'صفحة ترحيب';
  buttonDisabled: boolean = false;
  imageUrl: string = 'assets/logo.png';
}
```

```
// app.component.html
<h1>{{ pageTitle }}</h1>

<img [src]="imageUrl" alt="Logo">

<button [disabled]="buttonDisabled">إرسال</button>
```

شرح الكود

في هذا المثال، يتم نقل البيانات من ملف TypeScript إلى قالب HTML بثلاث طرق مختلفة:

- **الاستيفاء** `{{ pageTitle }}`: يعرض قيمة المتغير `pageTitle` كنص داخل وسم `<h1>`.
- **ربط الخاصية** `"imageUrl"=[src]`: يربط قيمة الخاصية `src` للصورة بقيمة المتغير `imageUrl`.
- **ربط الخاصية** `"buttonDisabled"=[disabled]`: يتحكم في حالة تفعيل أو تعطيل الزر بناءً على قيمة المتغير `buttonDisabled`.

خلاصة الدرس

- يتم ربط البيانات أحادي الاتجاه لنقل المعلومات من الكومبوننت إلى القالب.
- استخدم الاستيفاء `{{ }}` لإظهار النص مباشرة في القالب.
- استخدم ربط الخاصيات `[ ]` لتعيين قيم لخاصيات عناصر HTML.

القوالب وربط البيانات (Templates & Data Binding)

— ربط البيانات أحادي الاتجاه (One-Way Data)

(Binding): ربط الأحداث

ربط البيانات أحادي الاتجاه: ربط الأحداث

يُعد ربط الأحداث (Event Binding) جزءاً أساسياً من ربط البيانات أحادي الاتجاه في Angular. يتيح لك هذا النوع من الربط الاستجابة لتفاعلات المستخدمين على عناصر HTML في قالبك. عندما يقوم المستخدم بالضغط على زر، أو إدخال نص، أو تحريك الماوس، يمكنك تشغيل دالة (Function) في ملف TypeScript الخاص بالمكون لتنفيذ إجراء معين.

المفهوم الأساسي لربط الأحداث

يسمح لك ربط الأحداث بتوجيه البيانات أو الإشارات من القالب (Template) إلى المكون (Component). يتم تمثيل ربط الأحداث بواسطة الأقواس العادية (()) حول اسم الحدث (مثل `click` أو `change`). عند وقوع الحدث، يتم استدعاء الدالة المحددة داخل الأقواس.

مثال على ربط الأحداث

في هذا المثال، سنقوم بإنشاء زر يقوم بزيادة عداد عند النقر عليه.

ملف القالب (`app.component.html`):

```
<button (click)="incrementCounter()">زيادة العداد</button>
<p>العدد الحالي: {{ counter }}</p>
```

ملف المكون (`app.component.ts`):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
```



```
export class AppComponent {  
  counter: number = 0;  
  
  incrementCounter() {  
    this.counter++;  
  }  
}
```

شرح الكود

في المثال أعلاه، عندما يضغط المستخدم على الزر، يتم تفعيل حدث `click`. يقوم ربط الأحداث `(click)` باستدعاء الدالة `incrementCounter()` المعرفة في ملف TypeScript للمكون. تقوم هذه الدالة بزيادة قيمة المتغير `counter`، مما يؤدي إلى تحديث القيمة المعروضة في القالب.

- **الاتجاه:** يتم نقل البيانات من القالب إلى المكون (Template to Component).
- **الهدف:** الاستجابة لتفاعلات المستخدم مثل النقر، أو تحريك الماوس، أو إرسال نموذج.
- **الصيغة:** يتم استخدام الأقواس العادية `(event)=functionName()` لربط الأحداث.

القوالب وربط البيانات (Templates & Data Binding)

— ربط البيانات ثنائي الاتجاه (Two-Way Data Binding)

(Binding) باستخدام `ngModel`

html

ربط البيانات ثنائي الاتجاه (Two-Way Data Binding) باستخدام `ngModel`

يُعد ربط البيانات ثنائي الاتجاه (Two-Way Data Binding) من الميزات القوية في Angular التي تتيح مزامنة البيانات بين المكون (ملف TS) والقالب (ملف HTML) بشكل فوري ومستمر. فهو يجمع بين ربط الخاصية (Property Binding) وربط الحدث (Event Binding) في صيغة واحدة.

مفهوم `ngModel`

التوجيه (Directive) `ngModel` هو الأداة الرئيسية في Angular لتطبيق ربط البيانات ثنائي الاتجاه، ويُستخدم غالبًا مع عناصر إدخال النماذج (Form Inputs) مثل `<input>` و `<textarea>`. يسمح لك `ngModel` بربط قيمة حقل الإدخال بمتغير في كلاس المكون، بحيث:

- إذا تغير المتغير في الكلاس، تتغير القيمة في حقل الإدخال.
- إذا قام المستخدم بتغيير القيمة في حقل الإدخال، يتغير المتغير في الكلاس تلقائيًا.

صيغة "صندوق الموز" (Banana in a Box)

يتم تطبيق ربط البيانات ثنائي الاتجاه باستخدام الصيغة الخاصة `[(ngModel)]`، والتي تُعرف باسم "Banana in a Box" بسبب شكلها. هذه الصيغة اختصار لربط الخاصية `[(ngModel)]` وربط الحدث `(ngModelChange)` معًا.

مثال على الكود

ملف المكون (app.component.ts)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
})
export class AppComponent {
  userName: string = 'ضيف';
}
```

ملف القالب (app.component.html)

```
<h3>أهلاً بك، {{ userName }}</h3>

<input type="text" [(ngModel)]="userName" placeholder="اكتب اسمك هنا">
```

شرح الكود

في هذا المثال، قمنا بربط المتغير `userName` الموجود في ملف TS بحقل الإدخال في ملف HTML باستخدام `[(ngModel)]`. عندما يكتب المستخدم اسمه في حقل الإدخال، يتم تحديث المتغير `userName` في الحال. في نفس الوقت، يتم تحديث العنوان `<h3>` الذي يعرض قيمة `userName` بشكل فوري.

نقاط هامة

- **المزامنة التلقائية:** يوفر `ngModel` مزامنة فورية للبيانات بين الواجهة الأمامية (القالب) والمنطق الخلفي (المكون).
- **الاستخدام الأساسي:** يُستخدم بشكل أساسي في النماذج (Forms) لتسهيل التعامل مع مدخلات المستخدم.
- **تذكير:** لاستخدام `ngModel`، يجب استيراد `FormsModule` في ملف `app.module.ts` (أو ملف المكون المستقل `app.component.ts` إذا كنت تستخدم Standalone Components).

التوجيهات (Directives) — مقدمة إلى التوجيهات وأنواعها

مقدمة إلى التوجيهات وأنواعها

التوجيهات (Directives) هي إحدى الميزات الأساسية في Angular التي تتيح لك إضافة سلوكيات جديدة لعناصر HTML الموجودة في واجهة المستخدم (DOM). يمكن اعتبار التوجيهات بمثابة تعليمات خاصة لـ Angular تخبره بكيفية تغيير مظهر عنصر ما أو هيكله، أو إضافة وظيفة معينة إليه. إنها أساس بناء تطبيقات Angular الدينامية.

أنواع التوجيهات الأساسية

تنقسم التوجيهات في Angular إلى ثلاثة أنواع رئيسية، يتميز كل منها بوظيفة مختلفة:

- **توجيهات المكونات (Component Directives):** هذا هو النوع الأكثر شيوعاً. المكون (Component) هو في الأساس توجيه خاص يحتوي على قالب HTML مرتبط به (Template).
- **التوجيهات الهيكلية (Structural Directives):** تُستخدم لتغيير هيكل DOM عن طريق إضافة عناصر أو إزالتها بناءً على شروط معينة. مثل `*ngIf` و `*ngFor`.
- **توجيهات السمات (Attribute Directives):** تُستخدم لتغيير مظهر أو سلوك عنصر موجود بالفعل دون تغيير هيكله. مثل `[ngStyle]` و `[ngClass]`.

مثال على التوجيه الهيكلية (*ngIf)

```
<!-- app.component.html -->
<div *ngIf="isUserLoggedIn">
  مرحباً بك، لقد تم تسجيل دخولك بنجاح
</div>

<div *ngIf="!isUserLoggedIn">
  يرجى تسجيل الدخول لعرض المحتوى
</div>
```

شرح الكود

يوضح هذا المثال استخدام التوجيه الهيكلي `ngIf*`. يقوم Angular بتقييم تعبير `isUserLoggedIn`. إذا كانت النتيجة صحيحة (true)، يقوم Angular بإضافة العنصر الأول إلى DOM. إذا كانت النتيجة خاطئة (false)، فإنه يزيل العنصر الأول ويعرض العنصر الثاني بدلاً منه. هذا يسمح بالتحكم في العناصر التي تظهر للمستخدم بناءً على حالة التطبيق.

- التوجيهات هي تعليمات لتعديل واجهة المستخدم في Angular.
- توجيهات المكونات (Components) هي توجيهات ذات قوالب (Templates).
- التوجيهات الهيكلية (مثل `ngIf*`) تتحكم في بنية DOM (إضافة/إزالة عناصر).

التوجيهات (Directives) — التوجيهات الهيكلية

`ngIf`, `\*ngFor\*`:(Structural Directives)

التوجيهات الهيكلية (Structural Directives): `ngIf`, `\*ngFor\*`

التوجيهات الهيكلية (Structural Directives) في Angular هي أدوات قوية تسمح لنا بتغيير هيكل DOM (نموذج كائن المستند) لصفحة الويب ديناميكيًا. على عكس توجيهات السمات (Attribute Directives) التي تغير مظهر العناصر، فإن التوجيهات الهيكلية تقوم بإضافة عناصر أو إزالتها أو تكرارها بناءً على شروط منطقية.

مفهوم التوجيهات الهيكلية

التوجيه الهيكلية هو ببساطة تعليمات لـ Angular تخبره بكيفية بناء هيكل الصفحة. أهم توجيهين هيكلين هما: **`ngIf`** لإظهار أو إخفاء عنصر بناءً على شرط، و **`\*ngFor`** لتكرار عنصر عدة مرات بناءً على قائمة بيانات.

مثال على الكود

```
<!-- *ngIf: boolean إظهار/إخفاء عنصر بناءً على قيمة -->
<div *ngIf="isLoggedIn">
  <p>أهلاً بك، تم تسجيل دخولك بنجاح</p>
</div>

<!-- *ngFor: تكرار عنصر لكل عنصر في القائمة -->
<h3>قائمة المهام</h3>
<ul>
  <li *ngFor="let task of taskList">
    {{ task }}
  </li>
</ul>
```

شرح الكود

في هذا المثال، نستخدم توجيهين هيكليين:

1. `*ngIf="isUserLoggedIn"`: إذا كانت قيمة المتغير `isUserLoggedIn` صحيحة (`true`)، سيظهر عنصر `<div>` في الصفحة. أما إذا كانت خاطئة (`false`)، فسيقوم Angular بإزالة العنصر بالكامل من DOM، ولن يكون مرئيًا للمستخدم.

2. `*ngFor="let task of taskList"`: هذا التوجيه يقوم بإنشاء عنصر `<li>` واحد لكل عنصر موجود في مصفوفة `taskList`. هذا يسمح لنا بعرض قائمة بيانات ديناميكية دون كتابة HTML يدويًا لكل عنصر.

- `*ngIf` (التصيير الشرطي): يتحكم في وجود عنصر في DOM.
- `*ngFor` (التكرار): ينشئ نسخة من العنصر لكل عنصر في مجموعة البيانات.
- ملاحظة النجمة (`*`): تشير إلى أن التوجيه هيكلي (يغير بنية DOM).

التوجيهات (Directives) — التوجيهات الهيكلية

*(Structural Directives): *ngSwitch`

html

التوجيهات الهيكلية (Structural Directives): *ngSwitch

التوجيهات الهيكلية (Structural Directives) في Angular تغير هيكل DOM (Document Object Model) عن طريق إضافة العناصر أو إزالتها. يُستخدم التوجيه `*ngSwitch` لعرض محتوى معين بناءً على قيمة متغير، مما يوفر طريقة منظمة لإدارة حالات متعددة (multiple conditions) مشابهة لعبارة `switch` في البرمجة.

المفهوم الأساسي

يعمل `*ngSwitch` مع توجيهين آخرين: `*ngSwitchCase` و `*ngSwitchDefault`. يتم وضع `*ngSwitch` على العنصر الأب، وتحدد `*ngSwitchCase` العناصر التي سيتم عرضها بناءً على قيمة محددة. إذا لم تتطابق أي حالة، يتم عرض العنصر الذي يحمل `*ngSwitchDefault`.

مثال الكود

```
<!-- في المكون "userRole" افترض أن لدينا متغير -->
<div [ngSwitch]="userRole">
  <div *ngSwitchCase="'admin'">مرحباً أيها المدير! لديك صلاحيات كاملة</div>
  <div *ngSwitchCase="'editor'">أهلاً أيها المحرر. يمكنك تعديل المحتوى</div>
  <div *ngSwitchDefault>مرحباً بك كزائر. لديك صلاحيات محدودة</div>
</div>
```

شرح الكود

يقوم الكود بفحص قيمة المتغير `userRole` في المكون الأب. إذا كانت قيمة `userRole` هي `'admin'`، سيتم عرض العنصر الأول فقط. إذا كانت القيمة `'editor'`، سيتم عرض العنصر الثاني.

أما إذا لم تتطابق القيمة مع أي من الحالات المحددة، فسيتم عرض النص الافتراضي (Default) الموجود ضمن `*ngSwitchDefault`.

- يسمح `*ngSwitch` بعرض عنصر واحد فقط من مجموعة من العناصر بناءً على قيمة متغيرة.
- يجب استخدام `*ngSwitchCase` لتحديد حالات العرض المختلفة، و `*ngSwitchDefault` للحالة الافتراضية.
- يُعد `*ngSwitch` بديلاً أفضل وأكثر تنظيماً من استخدام توجيهات `*ngIf` و `*ngIf-else` المتعددة لنفس المتغير.

...

التوجيهات (Directives) — التوجيهات السيمائية

Attribute Directives: `[ngStyle]`, `[ngClass]`

التوجيهات السيمائية (Attribute Directives): `[ngClass]`, `[ngStyle]`

التوجيهات السيمائية (Attribute Directives) هي نوع من توجيهات Angular تُستخدم لتغيير مظهر أو سلوك عنصر HTML موجود. على عكس التوجيهات الهيكلية (Structural Directives) التي تغير بنية DOM، فإن التوجيهات السيمائية تعدل فقط سمات العنصر، مثل فئات CSS أو الأنماط المضمنة (inline styles). أهم توجيهين سمائيين هما `[ngClass]` و `[ngStyle]`.

مفهوم التوجيهات السيمائية

تسمح التوجيهات السيمائية بتطبيق تغييرات ديناميكية على العناصر. يسمح `[ngClass]` بتطبيق فئات CSS بناءً على شروط منطقية. بينما يسمح `[ngStyle]` بتعيين أنماط CSS المضمنة بشكل مباشر. هذه الأدوات تجعل واجهة المستخدم تتفاعل مع البيانات المتغيرة في التطبيق.

مثال تطبيقي: `[ngClass]` و `[ngStyle]`

```
<!-- HTML Template Code -->
<button (click)="toggleState()">Toggle State</button>

<div [ngClass]="{'is-active': isActive, 'warning-text': !isActive}"
      [ngStyle]="customStyle">
  This text changes dynamically.
</div>

// TypeScript Component Code
export class MyComponent {
  isActive: boolean = false;
  customStyle = { 'font-size': '16px', 'background-color': '#f0f0f0' };

  toggleState() {
    this.isActive = !this.isActive;
  }
}
```

```
if (this.isActive) {  
  this.customStyle.backgroundColor = '#d4edda'; // green background  
} else {  
  this.customStyle.backgroundColor = '#f8d7da'; // red background  
}  
}  
}
```

شرح الكود

يستخدم هذا المثال توجيه `[ngClass]` لتطبيق الفئة `is-active` إذا كان المتغير `isActive` صحيحًا (true)، ويطبق الفئة `warning-text` إذا كان خاطئًا (false). كما يستخدم توجيه `[ngStyle]` لربط الأنماط المضمنة (مثل لون الخلفية وحجم الخط) بمتغير `customStyle`. عند الضغط على الزر، تتغير قيمة `isActive`، وبالتالي يتغير مظهر العنصر بشكل فوري بناءً على الشروط الجديدة.

- `[ngClass]`: يتيح لك ربط فئات CSS بعناصر HTML بشكل مشروط (بناءً على الشروط المنطقية).
- `[ngStyle]`: يتيح لك تعيين أنماط CSS المضمنة ديناميكيًا بناءً على بيانات التطبيق.
- **التطبيق السيماتي:** لا يغير هذان التوجيهان بنية DOM (إضافة أو إزالة عناصر)، بل يعدلان مظهر العناصر الموجودة.

التوجيهات (Directives) — إنشاء توجيهات سماتية

مخصصة (Custom Attribute Directives)

html

إنشاء توجيهات سماتية مخصصة (Custom Attribute Directives)

تُعد التوجيهات (Directives) من المكونات الأساسية في Angular التي تتيح لنا تعديل سلوك أو مظهر عناصر HTML. في هذا الدرس، سنتعرف على كيفية إنشاء توجيهات سماتية (Attribute Directives) مخصصة، والتي تسمح لك بتغيير خصائص عنصر موجود بالفعل، مثل إضافة نمط معين أو تغيير لون الخلفية.

مفهوم التوجيهات السماتية المخصصة

التوجيه السيماتي (Attribute Directive) هو توجيه يتم تطبيقه كسمة (attribute) على عنصر HTML. على عكس المكونات (Components) التي تنشئ عرضًا خاصًا بها، تعمل التوجيهات السماتية على تعديل العنصر الذي يتم تطبيقها عليه، مما يتيح لك إعادة استخدام منطق معين (مثل إضافة تأثير بصري) بسهولة عبر أجزاء مختلفة من التطبيق.

مثال على توجيه سماتي لتغيير الخلفية

```
import { Directive, ElementRef, OnInit } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective implements OnInit {

  constructor(private el: ElementRef) {}

  ngOnInit() {
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }
}
```

```
}  
}
```

شرح الكود

يبدأ التوجيه باستخدام مزين (decorator) `@Directive`، ونحدد فيه المُحدد (selector) الخاص بنا كـ `'[appHighlight]'`. هذا يعني أن التوجيه سيتم تطبيقه على أي عنصر يحتوي على السمة `appHighlight`. نستخدم خدمة `ElementRef` للوصول إلى عنصر DOM الأصلي (الذي تم تطبيق التوجيه عليه)، وفي دالة `ngOnInit` نقوم بتعيين خاصية `backgroundColor` للعنصر إلى اللون الأصفر.

- **أهمية المُحدد السِماتي:** يجب استخدام الأقواس المربعة `[]` في المُحدد لتمييزه كتوجه سماتي، مما يجعله قابلاً للتطبيق كسمة HTML.
- **الوصول إلى العنصر:** نستخدم `ElementRef` للتعامل المباشر مع عنصر DOM الذي تم تطبيق التوجيه عليه، وتعديل خصائصه مثل الأنماط (styles) أو الفئات (classes).
- **إعادة الاستخدام:** تتيح لك التوجيهات السماتية تطبيق نفس المنطق على عناصر متعددة في أماكن مختلفة من التطبيق دون الحاجة لتكرار الكود.

الخدمات وحقن الاعتمادية (Services & Dependency Injection) — مفهوم الخدمات ولماذا نستخدمها

html

مفهوم الخدمات ولماذا نستخدمها

في تطبيقات Angular الكبيرة، نحتاج إلى فصل المنطق الخاص بالعمليات عن المكونات (Components). المكونات يجب أن تركز على عرض واجهة المستخدم فقط. الخدمات هي آليات نستخدمها لإنجاز المهام التي تحتاجها المكونات، مثل جلب البيانات من الخادم أو تخزين حالة التطبيق، مما يجعل الكود أكثر تنظيماً وقابلية لإعادة الاستخدام.

ما هي الخدمة (Service)؟

الخدمة في Angular هي فئة (class) تحتوي على منطق عمل محدد (مثل جلب البيانات، معالجة التاريخ، أو تخزين إعدادات المستخدم). دورها الأساسي هو توفير هذه الوظائف للمكونات المختلفة في التطبيق، مما يحافظ على نظافة المكونات ويزيد من كفاءة الكود.

مثال على إنشاء خدمة بسيطة

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class DataService {

  private datastore: string[] = ['Item 1', 'Item 2'];

  constructor() { }

  getData(): string[] {
```

```
return this.dataStore;
}

addData(item: string): void {
  this.dataStore.push(item);
}
}
```

شرح الكود

الكود أعلاه ينشئ خدمة اسمها `DataService`. السطر `@providedIn: 'root'` يجعل `Injectable` (`{` يخبر Angular أن هذه الفئة هي خدمة يمكن حقنها في أي مكان في التطبيق. الوظائف `getData()` و `addData()` تسمح للمكونات المختلفة في التطبيق بالوصول إلى نفس البيانات المشتركة وتعديلها.

- **فصل الاهتمامات (Separation of Concerns):** المكونات تركز على واجهة المستخدم، بينما الخدمات تتعامل مع منطق العمل.
- **إعادة استخدام الكود (Reusability):** يمكن حقن نفس الخدمة في عدة مكونات لاستخدام نفس الوظائف دون تكرار الكود.
- **إدارة الحالة (State Management):** تسمح الخدمات بمشاركة البيانات وتخزينها بين المكونات المختلفة.

الخدمات وحقن الاعتمادية (Services & Dependency Injection) — إنشاء خدمة باستخدام CLI

html

إنشاء خدمة باستخدام CLI

الخدمات (Services) في Angular تُستخدم لفصل منطق التطبيق (مثل جلب البيانات من الخادم أو إدارة الحالة) عن المكونات (Components). هذا يجعل المكونات أكثر نظافة ويسهل إعادة استخدام المنطق. CLI (واجهة سطر الأوامر) توفر أسهل طريقة لإنشاء هذه الخدمات.

المفهوم الأساسي

لإنشاء خدمة جديدة، نستخدم أمر `ng generate service`. هذا الأمر يقوم بإنشاء ملفات الخدمة المطلوبة تلقائيًا وتجهيزها لحقن الاعتمادية (Dependency Injection).

مثال الكود (CLI)

```
ng generate service data
```

شرح الكود

يخبر هذا الأمر واجهة سطر أوامر Angular بإنشاء خدمة جديدة. اسم الخدمة في هذا المثال هو "data". بعد تشغيل الأمر، سيتم إنشاء الملفات التالية في مجلد `src/app`:

- `data.service.ts`: هذا هو ملف الخدمة الرئيسي حيث نكتب منطق التطبيق.
- `data.service.spec.ts`: ملف الاختبار الخاص بالخدمة.

خلاصة الدرس

- استخدم `ng generate service` لإنشاء خدمة جديدة بسرعة.
- الخدمات تفصل المنطق البرمجي عن واجهة المستخدم.
- الخدمة التي يتم إنشاؤها تكون جاهزة للحقن في المكونات الأخرى.

...

الخدمات وحقن الاعتمادية (Services & Dependency Injection)

حقن الاعتمادية (Dependency Injection) — حقن الاعتمادية (Dependency Injection): مفهومها وكيفية عملها

html

حقن الاعتمادية (Dependency Injection): مفهومها وكيفية عملها

يُعد نمط حقن الاعتمادية (DI) أساسيًا في Angular. بدلاً من أن تقوم المكونات (Components) بإنشاء كائنات الخدمات (Services) التي تحتاجها، فإنها تعلن عن اعتماديتها عليها وتطلبها. يقوم نظام Angular بتوفير هذه الكائنات تلقائيًا. هذا يجعل الكود أكثر مرونة، ويسهل صيانته واختباره.

مفهوم حقن الاعتمادية (Dependency Injection)

المبدأ الأساسي لـ DI هو "انعكاس التحكم" (Inversion of Control). بدلاً من أن يتحكم المكون في إنشاء الخدمات، يقوم Angular (المُحقن) بالتحكم في عملية إنشاء الخدمات وتوفيرها للمكونات التي تطلبها. هذا يضمن أن كل مكون يحصل على مثيل (instance) واحد فقط من الخدمة، مما يحسن من كفاءة الذاكرة.

مثال برمجي

```
// 1. تعريف الخدمة (data.service.ts)
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  getData(): string[] {
    return ['بيانات 1', 'بيانات 2', 'بيانات 3'];
  }
}
```

```
// 2. حقن الخدمة في المكون (app.component.ts)
import { Component } from '@angular/core';
import { DataService } from '../data.service';

@Component({
  selector: 'app-root',
  template: `<ul><li *ngFor="let item of data">{{ item }}</li></ul>`
})
export class AppComponent {
  data: string[];

  constructor(private dataService: DataService) {
    this.data = this.dataService.getData();
  }
}
```

شرح الكود

في هذا المثال، لا يقوم المكون (AppComponent) بإنشاء نسخة جديدة من DataService. بدلاً من ذلك، يتم إعلان DataService كـ "اعتمادية" في باني المكون (constructor). عندما يتم إنشاء AppComponent، يقوم Angular تلقائيًا بالبحث عن مثيل (instance) لـ DataService وحقنه في المكون. يقوم المكون بعدها باستخدام وظيفة `getData()` من هذا المثيل الموفر.

ملخص الفكرة

- **فصل الاهتمامات:** حقن الاعتمادية يفصل بين المكونات والخدمات، حيث المكون لا يعرف كيفية إنشاء الخدمة، بل يطلبها فقط.
- **الباني (constructor) هو المفتاح:** يتم طلب الاعتماديات في Angular عن طريق تحديد نوعها في باني المكون.
- **المحاقن (Injectors):** نظام Angular يستخدم المحاقن (Injectors) لإدارة إنشاء وتوفير مثيلات الخدمات عند الحاجة.

الخدمات وحقن الاعتمادية (Services & Dependency Injection) — استخدام الخدمات في المكونات

استخدام الخدمات في المكونات

بعد إنشاء خدمة في Angular، نحتاج إلى ربط هذه الخدمة بالمكونات (Components) التي تحتاجها. يتم ذلك من خلال نظام حقن الاعتمادية (Dependency Injection) الخاص بـ Angular. هذه الطريقة تضمن أن المكونات تركز على مهمتها الأساسية (عرض البيانات)، بينما يتم تفويض المهام المعقدة (مثل جلب البيانات من الخادم) للخدمة.

كيفية حقن الخدمة (Service Injection)

الطريقة القياسية والأكثر شيوعًا لربط خدمة بمكون هي استخدام الدالة البانية (Constructor). يقوم Angular تلقائيًا بإنشاء نسخة واحدة (Singleton) من الخدمة وتزويد المكون بها عندما يطلبها في الدالة البانية.

مثال تطبيقي: ربط خدمة البيانات بمكون

لنفترض أن لدينا خدمة (DataService) تقوم بجلب قائمة من المنتجات. نريد أن يستخدم المكون (AppComponent) هذه الخدمة لعرض القائمة.

```
// app.component.ts

import { Component, OnInit } from '@angular/core';
import { DataService } from '../data.service'; // استيراد الخدمة

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
})
export class AppComponent implements OnInit {
  products: string[];
```

```
// حقن الخدمة في الدالة البانية 1.
constructor(private dataService: DataService) {
  this.products = []; // تهيئة المتغير
}

// استخدام الخدمة عند تهيئة المكون 2.
ngOnInit() {
  this.products = this.dataService.getProducts();
}
}
```

شرح الكود

في هذا المثال، قمنا باستيراد `DataService` أولاً. في الدالة البانية (`constructor`)، قمنا بتمرير متغير خاص (`private dataService: DataService`). عندما يرى Angular هذا التعريف، فإنه يفهم أن المكون يحتاج إلى نسخة من `DataService` ويقوم بتوفيرها تلقائيًا.

بعد ذلك، يمكننا استخدام `this.dataService` للوصول إلى أي دالة أو خاصية موجودة داخل الخدمة، كما حدث في الدالة `ngOnInit` حيث قمنا باستدعاء `getProducts()` لتعبئة مصفوفة المنتجات.

خلاصة الدرس

- **حقن الاعتمادية:** هو آلية Angular لتوصيل الخدمات بالمكونات.
- **الدالة البانية (Constructor):** هي المكان الصحيح لطلب الخدمة، حيث يتم تمرير الخدمة كمعامل (parameter).
- **نظافة الكود:** يؤدي استخدام الخدمات إلى فصل المنطق المعقد عن المكونات، مما يجعل الكود أكثر تنظيمًا وقابلية للاختبار.

الخدمات وحقن الاعتمادية (Services & Dependency Injection) — فهم الأنواع المختلفة لمزودي الخدمات (Providers)

html

فهم الأنواع المختلفة لمزودي الخدمات (Providers)

في Angular، مزود الخدمة (Provider) هو الوصفة التي تخبر نظام حقن الاعتمادية (DI) كيفية إنشاء نسخة من خدمة معينة. يمكننا تسجيل الخدمات بعدة طرق، تختلف باختلاف كيفية إنشاء النسخة (instance) ووقت إنشائها. فهم هذه الأنواع يسمح لنا بالتحكم في سلوك الخدمة وإمكانية استبدالها.

المفهوم الأساسي

عندما تطلب مكونات التطبيق خدمة معينة، يبحث Angular في المزودين المسجلين لمعرفة كيفية تقديم هذه الخدمة. أبسط طريقة هي استخدام `providedIn: 'root'` لإنشاء خدمة واحدة (Singleton) متاحة للتطبيق بأكمله. ولكن في بعض الأحيان، قد نحتاج إلى توفير فئة بديلة أو قيمة ثابتة، وهنا تبرز أهمية الأنواع المختلفة للمزودين مثل `useValue` و `useClass`.

مثال على المزود البديل (useClass)

```
// تعريف الخدمة الأصلية
// كرمز لطلب الخدمة (Logger) "injection token" نستخدم
import { InjectionToken } from '@angular/core';

export const LOGGER_TOKEN = new InjectionToken<Logger>('logger');

export abstract class Logger {
  abstract log(message: string): void;
}

// الفئة البديلة التي سنستخدمها فعليًا
export class ConsoleLogger implements Logger {
```

```

    log(message: string) {
      console.log(`[Console]: ${message}`);
    }
  }
}

// إعداد المزود في المكون أو الوحدة 3.
// ConsoleLogger سيتم توفيره، LOGGER_TOKEN عند طلب
@Component({
  selector: 'app-root',
  template: '...',
  providers: [
    { provide: LOGGER_TOKEN, useClass: ConsoleLogger }
  ]
})
export class AppComponent {
  constructor(@Inject(LOGGER_TOKEN) private logger: Logger) {
    this.logger.log("Service started");
  }
}

```

شرح الكود

في المثال، قمنا بإنشاء رمز (Token) باسم `LOGGER_TOKEN` لتمثيل الخدمة. ثم، في إعدادات المزود `providers`، نستخدم `useClass` لإخبار Angular أنه عندما يتم طلب `LOGGER_TOKEN`، يجب عليه استخدام الفئة `ConsoleLogger` لإنشاء نسخة الخدمة. هذا يسمح لنا بتغيير طريقة تنفيذ الخدمة في المستقبل (مثل استبدالها بـ `MockLogger` للاختبار) دون تعديل الكود الذي يستخدم الخدمة.

- `providedIn: 'root'`: الطريقة الافتراضية لإنشاء نسخة واحدة للخدمة على مستوى التطبيق (Singleton).
- `useClass`: يتيح لك تحديد فئة بديلة ليتم استخدامها بدلاً من الفئة الأصلية عند حقن الخدمة.
- `useValue`: يوفر قيمة ثابتة (كائن أو متغير) مباشرة بدلاً من إنشاء نسخة جديدة من الفئة.

التوجيه (Routing) — مفهوم التوجيه في تطبيقات الصفحة الواحدة (SPA)

html

مفهوم التوجيه في تطبيقات الصفحة الواحدة (SPA)

في تطبيقات Angular الحديثة، غالبًا ما نحتاج إلى عرض محتوى مختلف للمستخدمين بناءً على عنوان URL في شريط المتصفح (على سبيل المثال، الانتقال من الصفحة الرئيسية إلى صفحة "حول"). التوجيه (Routing) هو الآلية التي تسمح بتنفيذ هذا التنقل السلس دون الحاجة لإعادة تحميل الصفحة بالكامل، وهو السمة المميزة لتطبيقات الصفحة الواحدة (SPA).

ما هو التوجيه (Routing) في Angular؟

التوجيه هو نظام يسمح لنا بتعيين عناوين URL محددة (مسارات) لمكونات (Components) معينة داخل التطبيق. بدلاً من جلب صفحة جديدة من الخادم عند كل نقرة، يقوم التوجيه في Angular بتغيير المحتوى المعروض داخل جزء محدد من الصفحة الحالية (يُعرف بـ `<router-outlet>`) بناءً على المسار المحدد.

مثال إعداد التوجيه الأساسي

```
// app-routing.module.ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { AboutComponent } from './about/about.component';

const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '', redirectTo: '/home', pathMatch: 'full' }
];

@NgModule({
```



```
imports: [RouterModule.forRoot(routes)],
exports: [RouterModule]
})
export class AppRoutingModule { }
```

شرح الكود

في هذا المثال، نقوم بتعريف مصفوفة من المسارات (`routes`). كل كائن داخل المصفوفة يربط بين مسار محدد في المتصفح (مثل `'home'`) ومكون (`HomeComponent`) Angular يجب عرضه عند زيارة هذا المسار. المسار الافتراضي (`'/'`) يقوم بإعادة توجيه المستخدم إلى صفحة `'home/'` عند فتح التطبيق لأول مرة. يقوم `RouterModule.forRoot(routes)` بتكوين الموجه بناءً على هذه التعريفات.

- **التنقل السريع:** يتيح التوجيه للمستخدم التنقل بين العروض المختلفة في التطبيق دون إعادة تحميل المتصفح، مما يوفر تجربة مستخدم سريعة.
- **المسارات والمكونات:** يتم ربط كل مسار (URL) بمكون معين، ويتم عرض المكون المقابل في منطقة `<router-outlet>`.
- **إدارة حالة المتصفح:** يستخدم Angular Routing واجهة برمجة تطبيقات تاريخ المتصفح (History API) لتحديث عنوان URL بشكل صحيح والحفاظ على وظيفة أزرار "الخلف" و"الأمام" في المتصفح.

التوجيه (Routing) — إعداد وحدة التوجيه (App Routing Module)

إعداد وحدة التوجيه (App Routing Module)

لكي يتمكن تطبيق Angular من التنقل بين الصفحات المختلفة، يجب علينا أولاً تحديد المسارات (Routes) التي تربط كل مسار URL بالمكون (Component) المسؤول عن عرضه. وحدة التوجيه الرئيسية (App Routing Module) هي المكان المركزي لتعريف هذه المسارات في التطبيق.

تعريف المسارات الرئيسية

يتم تعريف المسارات في مصفوفة من النوع `Routes`. تحتوي هذه المصفوفة على كائنات، يحدد كل كائن المسار (path) والمكون (component) المقابل له. عند بدء تشغيل التطبيق، يتم تسجيل هذه المسارات باستخدام الدالة `RouterModule.forRoot()`.

مثال على App Routing Module

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';
import { HomeComponent } from '../home/home.component';
import { AboutComponent } from '../about/about.component';

const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '', redirectTo: '/home', pathMatch: 'full' }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
```

```
export class AppRoutingModuleModule { }
```

شرح الكود

في هذا المثال، قمنا بتعريف مصفوفة `routes`. الكائن الأول يربط المسار `'home'` بالمكون `HomeComponent`، والكائن الثاني يربط `'about'` بالمكون `AboutComponent`. السطر الأخير `redirectTo` يضمن توجيه المستخدم تلقائيًا إلى صفحة `'home'` عند فتح التطبيق. يتم تسجيل هذه المسارات في الوحدة باستخدام `RouterModule.forRoot(routes)`.

نقاط أساسية

- وحدة التوجيه الرئيسية هي المخطط العام لمسارات التطبيق.
- يتم تعريف المسارات في مصفوفة `Routes` تربط المسار بالمكون المقابل.
- يستخدم `RouterModule.forRoot()` لتسجيل المسارات في الوحدة الرئيسية للتطبيق.

التوجيه (Routing) — تعريف المسارات (Routes) والتوجيه الأساسي

تعريف المسارات (Routes) والتوجيه الأساسي

يعد التوجيه (Routing) جزءًا أساسيًا في تطبيقات Angular، فهو يسمح للمستخدمين بالتنقل بين الصفحات المختلفة عن طريق تغيير عنوان URL. يقوم نظام التوجيه بربط كل عنوان URL بمكون (Component) محدد، مما يسمح بعرض محتوى مختلف دون الحاجة إلى إعادة تحميل الصفحة بالكامل.

مصفوفة المسارات الأساسية (Routes Array)

في Angular، يتم تعريف المسارات عن طريق مصفوفة تُسمى `routes`. كل عنصر في هذه المصفوفة هو كائن يحدد المسار (`path`) والمكون المرتبط به (`component`). هذه المصفوفة هي خريطة توجيه التطبيق.

مثال على تعريف المسارات

```
// app-routing.module.ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from '../home/home.component';
import { AboutComponent } from '../about/about.component';

const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '', redirectTo: '/home', pathMatch: 'full' }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
```

```
})  
export class AppRoutingModule { }
```

شرح الكود

يحتوي هذا الكود على مصفوفة `routes` تُعرّف مسارات التطبيق. المسار الأول يربط العنوان `home/` بالمكون `HomeComponent`، والمسار الثاني يربط `about/` بالمكون `AboutComponent`. المسار الثالث يمثل إعادة توجيه: إذا كان المسار فارغاً (`''`)، فسيتم تحويل المستخدم تلقائيًا إلى `home/`.

- **المسار (path):** هو الجزء الموجود في شريط عنوان المتصفح الذي يحدد المورد المطلوب.
- **المكون (component):** هو الواجهة المرئية التي يتم عرضها عند مطابقة المسار.
- **إعادة التوجيه (redirectTo):** تستخدم لتوجيه المستخدم من مسار معين إلى مسار آخر.

التوجيه (Routing) — الروابط الموجهة (Router)

Links و`router-outlet`

html

الروابط الموجهة (Router Links) و`router-outlet`

في بناء تطبيقات Angular، نحتاج إلى طريقة للتنقل بين المكونات المختلفة (الصفحات) دون الحاجة لإعادة تحميل الصفحة بالكامل. يوفر نظام التوجيه (Routing) في Angular هذه الآلية، حيث نستخدم الروابط الموجهة (`routerLink`) ووسم `router-outlet` لإدارة عرض المحتوى بشكل ديناميكي.

مفهوم `routerLink` و`router-outlet`

يعد router-outlet هو المكون الأساسي في التوجيه، فهو بمثابة حاوية ديناميكية يتم فيها تحميل المكونات بناءً على المسار الحالي. أما routerLink فهو سمة (attribute) تُستخدم بدلاً من href في وسم ، حيث تخبر Angular بالانتقال إلى مسار معين دون تحديث الصفحة.

مثال على الاستخدام

```
<!-- app.component.html -->
<nav>
  <a routerLink="/home">الصفحة الرئيسية</a>
  <a routerLink="/about">حول التطبيق</a>
  <a routerLink="/contact">اتصل بنا</a>
</nav>

<router-outlet></router-outlet>
```

شرح الكود

في المثال أعلاه، يتم استخدام routerLink لتحديد المسار الذي سيتم الانتقال إليه عند النقر على الرابط (مثل `/home`). عند النقر، يقوم Angular بتحميل المكون المرتبط بهذا المسار (والذي قمنا بتعريفه في

ملف التوجيه (app-routing.module.ts) وعرضه داخل المكان المخصص له وهو وسم router-outlet.

- **`routerLink` (الرابط الموجه):** يُستخدم لتوجيه المستخدمين داخليًا بين المكونات في Angular، ويحل محل السمة href التقليدية.
- **`router-outlet` (منفذ الموجه):** هو المكان الذي تُعرض فيه المكونات عند تغيير المسار، ويعمل كحاوية ديناميكية.
- **التنقل السريع:** يتيح `routerLink` التنقل بين المكونات دون إعادة تحميل الصفحة بالكامل، مما يحسن تجربة المستخدم.

التوجيه (Routing) — تمرير المعلومات في المسارات (Route Parameters)

تمرير المعلومات في المسارات (Route Parameters)

في تطبيقات Angular المعقدة، نحتاج غالبًا إلى تمرير بيانات محددة ضمن المسار (URL) لتحديد المورد الذي سيتم عرضه. على سبيل المثال، عند عرض تفاصيل منتج معين، نحتاج إلى تمرير رقم تعريفه (ID) في عنوان URL. هذا هو دور معلومات المسار (Route Parameters).

مفهوم المعلومات الديناميكية

معلومات المسار هي أجزاء ديناميكية في المسار يتم تعريفها بعلامة النقطتين (:) متبوعة باسم المعلمة. هذا يسمح لنا بإنشاء مسار واحد يمكنه استيعاب قيم متعددة، مما يجعله أكثر كفاءة من إنشاء مسار ثابت لكل عنصر.

مثال برمجي

لنفترض أن لدينا مسارًا لعرض تفاصيل منتج، ونريد تمرير رقم تعريفه (ID):

```
// 1. تعريف المسار في app-routing.module.ts
const routes: Routes = [
  { path: 'products/:id', component: ProductDetailsComponent }
];

// 2. قراءة المعلمة في المكون (ProductDetailsComponent)
import { ActivatedRoute } from '@angular/router';

export class ProductDetailsComponent implements OnInit {
  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    this.route.paramMap.subscribe(params => {
      const productId = params.get('id');
      console.log('Product ID:', productId);
    });
  }
}
```



```
}  
}
```

شرح الكود

في ملف التوجيه، قمنا بتعريف مسار ديناميكي (`products/:id`) حيث يمثل `id` معلمة متغيرة. في المكون (`ProductDetailsComponent`)، قمنا بحقن خدمة `ActivatedRoute`. نستخدم `paramMap` للاشتراك في التغييرات والحصول على قيمة المعلمة `id` من عنوان URL. يتيح لنا هذا تحميل البيانات الخاصة بالمنتج المحدد.

- **تعريف المعلمة:** يتم تحديد المعلومات الديناميكية في تعريف المسار باستخدام الرمز `:` متبوعًا باسم المعلمة.
- **قراءة المعلمة:** نستخدم خدمة `ActivatedRoute` داخل المكون للوصول إلى بيانات المسار الحالي، بما في ذلك المعلومات.
- **استخدام `paramMap`:** يفضل استخدام `paramMap` كـ `Observable` للاستجابة للتغييرات الديناميكية في المعلومات دون الحاجة لإعادة تحميل المكون.

التوجيه (Routing) — الحراس (Route Guards) للحماية (مقدمة بسيطة)

الحراس (Route Guards) للحماية (مقدمة بسيطة)

في بناء تطبيقات Angular، نحتاج في كثير من الأحيان لحماية بعض المسارات (Routes) ومنع المستخدمين من الوصول إليها إلا بعد استيفاء شروط معينة. على سبيل المثال، منع المستخدم غير المسجل دخوله من الوصول إلى صفحة "لوحة التحكم". حراس التوجيه (Route Guards) هي أدوات في Angular تسمح لنا بتنفيذ هذه الحماية.

مفهوم CanActivate

CanActivate هو أشهر أنواع الحراس وأكثرها استخداماً. وظيفته الأساسية هي تحديد ما إذا كان يمكن تفعيل مسار معين والسماح بالوصول إليه أم لا. يقوم الحارس بتنفيذ منطق معين (مثل التحقق من حالة تسجيل الدخول) قبل تحميل المسار. إذا أعاد الحارس القيمة **true**، يتم السماح بالتوجيه؛ وإذا أعاد **false**، يتم حظر الوصول.

مثال بسيط لتطبيق الحماية

```
// auth.guard.ts (إنشاء الحارس 1)
import { inject } from '@angular/core';
import { CanActivateFn } from '@angular/router';

export const authGuard: CanActivateFn = (route, state) => {
  // هنا يتم التحقق من حالة المستخدم (مثلاً، هل هو مسجل للدخول؟)
  // مثال: لنفترض أنه مسجل للدخول
  const isAuthenticated = true;

  if (isAuthenticated) {
    return true; // السماح بالوصول إلى المسار
  } else {
    // يمكن هنا إعادة توجيه المستخدم لصفحة تسجيل الدخول
    return false; // منع الوصول
  }
};
```

```
// 2. تطبيق الحارس على المسار في ملف التوجيه (app-routing.module.ts)
import { Routes } from '@angular/router';
import { authGuard } from './auth.guard';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'dashboard', component: DashboardComponent, canActivate: [authGuard] },
  { path: 'login', component: LoginComponent }
];
```

شرح الكود

في المثال أعلاه، قمنا بإنشاء حارس بسيط (`authGuard`) يستخدم وظيفة `CanActivateFn`. يتحقق الحارس من المتغير `isAuthenticated`. إذا كان `true`، يسمح بالوصول إلى صفحة لوحة التحكم (Dashboard). يتم تطبيق هذا الحارس على المسار `'dashboard'` باستخدام الخاصية `canActivate` في تعريف المسار.

- **حماية المسارات:** الحراس هي خط الدفاع الأول لمنع الوصول غير المصرح به إلى صفحات معينة في التطبيق.
- **التحكم في التوجيه:** يمكن استخدام الحراس لفرض شروط قبل السماح بالانتقال بين الصفحات أو حتى قبل تحميل بيانات معينة.
- **تحسين الأمان:** تساهم الحراس في زيادة أمان التطبيق عن طريق ضمان أن المستخدمين لديهم الأذونات اللازمة قبل عرض المحتوى الحساس.

النماذج (Forms) — مقدمة إلى نماذج Angular

html

مقدمة إلى نماذج Angular (Angular Forms)

تعتبر النماذج (Forms) جزءًا أساسيًا في أي تطبيق ويب تفاعلي. تتيح لك نماذج Angular جمع مدخلات المستخدم (مثل أسماء المستخدمين، كلمات المرور، أو تحديد الخيارات) ومعالجتها بكفاءة. توفر Angular آليتين رئيسيتين للتعامل مع النماذج: النماذج القائمة على القوالب (Template-Driven Forms) والنماذج التفاعلية (Reactive Forms).

ما هي نماذج Angular؟

نموذج Angular هو مجموعة من عناصر التحكم (Controls) التي تسمح للمستخدم بإدخال البيانات. وظيفة Angular هي ربط هذه العناصر بمتغيرات في شيفرة TypeScript، مما يتيح لك الوصول إلى البيانات المدخلة والتحقق من صحتها (Validation) قبل إرسالها إلى الخادم. يختار المطورون بين النوعين حسب تعقيد النموذج وحجم البيانات.

مثال على نموذج بسيط (نموذج قائم على القوالب)

```
<form (ngSubmit)="onSubmit()" #userForm="ngForm">
  <div>
    <label for="username">اسم المستخدم</label>
    <input type="text" id="username" name="username" [(ngModel)]="model.user" />
  </div>

  <button type="submit" [disabled]="!userForm.valid">إرسال</button>
</form>
```

شرح الكود

هذا المثال يوضح استخدام النماذج القائمة على القوالب. `(ngSubmit)` هو حدث يتم تشغيله عند الضغط على زر الإرسال. `[(ngModel)]` هو ربط ثنائي الاتجاه يربط قيمة حقل الإدخال بمتغير `model.username` في TypeScript. السمة `required` تُطبق قواعد التحقق من الصحة، و `userForm="ngForm#"` ينشئ مرجعًا للنموذج بالكامل. يتم استخدام `userForm.valid!=[disabled]` لتعطيل زر الإرسال إذا كان النموذج غير صالح (مثل عدم تعبئة الحقل المطلوب).

ملخص المفاهيم الرئيسية

- تُستخدم نماذج Angular لجمع البيانات من المستخدم وإدارتها داخل التطبيق.
- هناك نوعان رئيسيان من النماذج: النماذج التفاعلية (Reactive) والنماذج القائمة على القوالب (Template-Driven).
- توفر Angular أدوات لربط البيانات والتحقق من صحتها بسهولة عبر توجيهات (Directives) مثل `ngSubmit` و `ngModel`.

النماذج (Forms) — النماذج القائمة على القوالب (Template-Driven Forms): الأساسيات

و`FormsModule`

النماذج القائمة على القوالب (Template-Driven Forms): الأساسيات و`FormsModule`

تُعد النماذج القائمة على القوالب (Template-Driven Forms) إحدى الطريقتين الرئيسيتين لبناء النماذج في Angular. في هذا الأسلوب، يتم تعريف منطق النموذج ومعالجة البيانات بشكل أساسي في ملف HTML (القالب) باستخدام التوجيهات (Directives) الخاصة بـ Angular. وهو الخيار الأفضل للنماذج البسيطة حيث يتم التعامل مع معظم المنطق مباشرة في القالب.

أساسيات `FormsModule` و`ngModel`

لاستخدام النماذج القائمة على القوالب، يجب أولاً استيراد وحدة **FormsModule** في ملف الوحدة (Module) الخاص بالتطبيق. هذه الوحدة توفر توجيهات أساسية مثل **ngModel** لربط حقول الإدخال ببيانات المكون بطريقة ثنائية الاتجاه، و**ngSubmit** للتعامل مع إرسال النموذج. يعتبر **[(ngModel)]** (ربط صندوق الموز) هو المفتاح لربط حقل الإدخال بالبيانات.

مثال على الكود

```
<!-- app.component.html -->
<form (ngSubmit)="onSubmit()" #userForm="ngForm">
  <div>
    <label for="username">اسم المستخدم</label>
    <input type="text" id="username" name="username" [(ngModel)]="user.username" />
  </div>
  <button type="submit">تسجيل</button>
</form>
```

شرح الكود

يتم استيراد `FormsModule` في ملف الوحدة لتمكين التوجيهات المستخدمة في القالب. التوجيه `[(ngModel)]` يقوم بإنشاء ربط ثنائي الاتجاه بين حقل الإدخال (Input) والمتغير `user.username` في المكون، مما يعني أن أي تغيير في الحقل يعكس تلقائيًا على المتغير والعكس صحيح. التوجيه `"userForm="ngForm#" (ngSubmit)` يُنشئ متغير قالب محلي يمثل النموذج بأكمله، بينما `onSubmit` يستدعي الدالة عند النقر على زر الإرسال.

- تعتمد النماذج القائمة على القوالب بشكل أساسي على التوجيهات في ملف HTML.
- يجب استيراد `FormsModule` لتمكين هذا النوع من النماذج.
- يُستخدم `[(ngModel)]` لربط حقول الإدخال بالبيانات في المكون (Two-Way Data Binding).

النماذج (Forms) — النماذج القائمة على القوالب: التحقق من الصحة (Validation)

html

النماذج القائمة على القوالب: التحقق من الصحة (Validation)

في تطوير تطبيقات الويب، من الضروري التحقق من صحة البيانات التي يدخلها المستخدم. تتيح لنا Angular إضافة قواعد التحقق (Validation) مباشرة إلى عناصر الإدخال (inputs) في نموذج HTML القائم على القوالب، مما يضمن أن البيانات تلبى المتطلبات قبل إرسال النموذج.

توجيهات التحقق المضمنة (Built-in Validation Directives)

تستخدم Angular سمات HTML القياسية للتحقق من الصحة. عند إضافة سمة مثل `required` (مطلوب) أو `minlength` (الحد الأدنى للطول) إلى حقل إدخال، تقوم Angular تلقائيًا بمراقبة حالة صلاحية هذا الحقل وتحديث حالة النموذج الكلية. يمكننا بعد ذلك استخدام هذه الحالة لعرض رسائل الأخطاء للمستخدم.

مثال على التحقق الأساسي

```
<form #userForm="ngForm" (ngSubmit)="onSubmit(userForm)">
  <div>
    <label for="username">اسم المستخدم</label>
    <input type="text"
      id="username"
      name="username"
      ngModel
      required
      #username="ngModel">
    <div *ngIf="username.invalid && username.touched">
      <p>!اسم المستخدم مطلوب</p>
    </div>
  </div>
```



```
<button type="submit" [disabled]="userForm.invalid">إرسال</button>  
</form>
```

شرح الكود

في المثال أعلاه، استخدمنا السمة `required` لجعل حقل إدخال اسم المستخدم إلزاميًا. المتغير المحلي `"username"=ngModel#` يتيح لنا الوصول إلى حالة التحقق الخاصة بهذا الحقل. نستخدم التوجيه `ngIf*` لعرض رسالة الخطأ فقط عندما يكون الحقل غير صالح (`username.invalid`) وقد تفاعل المستخدم معه (`username.touched`). كما قمنا بتعطيل زر الإرسال باستخدام `"userForm.invalid"=[disabled]` حتى يصبح النموذج بالكامل صالحًا.

- **التحقق باستخدام السمات:** يتم التحقق من صحة النماذج القائمة على القوالب عن طريق إضافة سمات HTML مباشرة إلى عناصر الإدخال.
- **مراقبة الحالة:** نستخدم متغيرًا محليًا (مثل `"username"=ngModel#`) للوصول إلى حالة التحقق الخاصة بحقل معين ومراقبة خصائصه مثل `invalid` و `touched`.
- **عرض رسائل الخطأ:** يتم عرض رسائل الخطأ بشكل شرطي باستخدام `ngIf*` بناءً على حالة الحقل لضمان تجربة مستخدم جيدة.

النماذج (Forms) — النماذج التفاعلية (Reactive Forms) (Forms): `FormControl`, `FormGroup`

النماذج التفاعلية (Reactive Forms): `FormControl` و `FormGroup`

تُعد النماذج التفاعلية (Reactive Forms) منهجية قوية لإدارة مدخلات المستخدم في Angular. بدلاً من الاعتماد على قالب HTML للتحكم في تدفق البيانات، يتم إنشاء نموذج البيانات بالكامل برمجياً داخل ملف TypeScript الخاص بالمكون، مما يوفر تحكماً دقيقاً في الحالة والتحقق من الصحة.

مفهوم `FormControl`

يمثل `FormControl` حقل إدخال واحد في النموذج (مثل حقل الاسم أو البريد الإلكتروني). يقوم هذا الكائن بتتبع قيمة الحقل وحالة التحقق من صحته، مما يسمح لنا بالوصول إلى قيمته وتعديلها بسهولة من خلال الكود.

مفهوم `FormGroup`

يستخدم `FormGroup` لتجميع عدة حقول `FormControl` معاً. عندما نقوم بتجميع الحقول في `FormGroup`، يمكننا التعامل معها كوحدة واحدة. يقوم `FormGroup` بتتبع القيمة الإجمالية لجميع الحقول بداخله، ويحدد الحالة العامة للنموذج (سواء كان صحيحاً أو خاطئاً).

مثال كود بسيط

```
import { FormControl, FormGroup } from '@angular/forms';

export class UserProfileComponent {
  userForm = new FormGroup({
    firstName: new FormControl(''),
    lastName: new FormControl(''),
    email: new FormControl('')
  });
}
```

```
});  
}
```

شرح الكود

في هذا المثال، قمنا بإنشاء كائن `FormGroup` باسم `userForm`. يحتوي هذا الكائن على ثلاثة حقول فردية (من نوع `FormControl`) وهي `firstName` و `lastName` و `email`. يتيح لنا هذا الهيكل إدارة النموذج ككل وتعديل قيم الحقول الفردية من خلال كائن `userForm` الرئيسي.

- **`FormControl`**: يمثل حقل إدخال واحد، ويتتبع قيمته وحالة التحقق الخاصة به.
- **`FormGroup`**: يجمع عدة حقول `FormControl` معاً، ويدير حالتها الإجمالية.
- **التحكم البرمجي**: يتيح لنا النماذج التفاعلية إنشاء النماذج والتحقق من صحتها بالكامل داخل ملف TypeScript.

النماذج (Forms) — النماذج التفاعلية: `FormArray`

النماذج التفاعلية: `FormArray`

في بناء النماذج التفاعلية (Reactive Forms) باستخدام Angular، قد تواجه الحاجة إلى إنشاء حقول قابلة للتكرار أو قوائم ديناميكية. على سبيل المثال، نموذج يسمح للمستخدم بإضافة أكثر من رقم هاتف أو بريد إلكتروني. بدلاً من تحديد عدد ثابت من الحقول مسبقاً، يتيح لك `FormArray` التعامل مع هذه القوائم بمرونة.

مفهوم `FormArray`

`FormArray` هو حاوية (container) لمجموعة من عناصر التحكم (Form Controls) أو مجموعات التحكم (Form Groups) التي يمكن إدارتها كقائمة. إنها أداة أساسية لإنشاء حقول قابلة للإضافة والحذف ديناميكياً من واجهة المستخدم. يتميز `FormArray` بكونه مرناً، حيث يمكن إضافة عناصر جديدة إليه (push) أو حذف عناصر موجودة منه (removeAt) بسهولة.

مثال على الكود

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, FormArray, Validators } from '@angular/forms';

@Component({ ... })
export class MyFormComponent {
  myForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.myForm = this.fb.group({
      userName: ['', Validators.required],
      skills: this.fb.array([this.newSkillGroup()]) // FormArray تهيئة
    });
  }

  // سهولة الوصول FormArray دالة للحصول على
  get skillsArray(): FormArray {
    return this.myForm.get('skills') as FormArray;
  }
}
```

```

}

// للمهارة الواحدة (FormGroup) دالة لإنشاء مجموعة حقول جديدة
newSkillGroup(): FormGroup {
  return this.fb.group({
    skillName: ['', Validators.required],
    yearsExperience: [0]
  });
}

// دالة لإضافة مجموعة حقول جديدة إلى FormArray
addSkill(): void {
  this.skillsArray.push(this.newSkillGroup());
}

// دالة لحذف حقل من القائمة بناءً على المؤشر (index)
removeSkill(index: number): void {
  this.skillsArray.removeAt(index);
}
}

```

شرح الكود

في هذا المثال، قمنا بإنشاء نموذج (FormGroup) يحتوي على حقل عادي ('userName') وقائمة ديناميكية ('skills'). القائمة 'skills' هي 'FormArray'. يتم تهيئة هذا المصفوفة في البداية بعنصر واحد (group)، ثم نستخدم الدالة 'addSkill()' لإضافة مجموعات حقول جديدة بشكل ديناميكي إلى 'skillsArray' عند ضغط المستخدم على زر "إضافة مهارة". الدالة 'removeSkill()' تسمح بحذف عنصر معين من القائمة.

- 'FormArray' هو الأداة المثالية لإدارة قوائم الحقول المتكررة (مثل إضافة أكثر من بريد إلكتروني أو رقم هاتف).
- يُستخدم مع 'FormBuilder' لإنشاء مجموعات حقول قابلة للإضافة والحذف ديناميكيًا.
- يمكنك إضافة عناصر جديدة إلى 'FormArray' باستخدام الدالة 'push()' وحذفها باستخدام 'removeAt()'.

النماذج (Forms) — النماذج التفاعلية: التحقق من الصحة المخصص (Custom Validation)

النماذج التفاعلية: التحقق من الصحة المخصص (Custom Validation)

عند بناء النماذج في Angular، غالباً ما نحتاج إلى قواعد تحقق من الصحة أكثر تعقيداً من القواعد الأساسية المدمجة (مثل required أو minLength). التحقق المخصص (Custom Validation) يسمح لنا بإنشاء وظائف تحقق خاصة بنا لتطبيق قواعد عمل فريدة (مثل التحقق من تنسيق بريد إلكتروني خاص، أو التأكد من تطابق كلمتي مرور).

مفهوم التحقق المخصص

وظيفة التحقق المخصص هي دالة تأخذ كائن `AbstractControl` (وهو الكائن الذي يمثل حقل النموذج) كمدخل. إذا كان الحقل صالحاً، يجب أن تعيد الدالة `null`. أما إذا كان غير صالح، فيجب أن تعيد كائن خطأ يحتوي على مفتاح يصف الخطأ (مثل `{ passwordMismatch: true }`).

مثال على كود التحقق المخصص

```
import { AbstractControl, ValidatorFn } from '@angular/forms';

/**
 * دالة للتحقق من أن قيمة الحقل هي رقم ضمن نطاق معين.
 * @param min الحد الأدنى المسموح به.
 * @param max الحد الأقصى المسموح به.
 */
export function rangeValidator(min: number, max: number): ValidatorFn {
  return (control: AbstractControl): { [key: string]: any } | null => {
    const value = control.value;
    // وأنها خارج النطاق المحدد null للتحقق من أن القيمة ليست
    if (value !== null && (value < min || value > max)) {
      return { 'range': { min: min, max: max, actual: value } };
    }
    return null;
  };
}
```

```
};  
}
```

شرح الكود

يُنشئ هذا المثال دالة `rangeValidator` للتحقق من أن قيمة حقل معين تقع ضمن نطاق عددي محدد. تقبل الدالة `min` و `max` كمدخلات. تقوم الدالة الداخلية بالتحقق من قيمة الحقل (`control.value`). إذا كانت القيمة خارج النطاق، تُعيد الدالة كائن خطأ يحتوي على مفتاح `range`. إذا كانت القيمة ضمن النطاق، تُعيد `null`، مما يشير إلى أن الحقل صالح. يتم استخدام هذه الدالة عند تعريف `FormControl` في نموذجنا التفاعلي.

- **المرونة:** يسمح التحقق المخصص بتطبيق أي منطق تحقق معقد لا توفره Angular بشكل افتراضي.
- **البنية:** الدالة تعيد `null` إذا كان الحقل صالحاً، وكائن خطأ إذا كان غير صالح.
- **الاستخدام:** يتم تمرير الدالة المخصصة إلى مصفوفة المحققين (`validators`) عند تعريف `FormControl` أو `FormGroup`.

عميل (HTTP Client) HTTP — مقدمة لطلب البيانات من الخوادم

html

مقدمة لطلب البيانات من الخوادم

في تطبيقات الويب الحديثة، نادرًا ما تكون البيانات ثابتة. في أغلب الأحيان، يحتاج تطبيق Angular (الذي يعمل كـ "عميل") إلى جلب بيانات حية وديناميكية من خادم بعيد (API). هذه العملية تتم باستخدام بروتوكول HTTP. في هذا الدرس، نتعرف على كيفية استخدام خدمة `HttpClient` في Angular لإرسال طلبات لجلب البيانات.

خدمة HttpClient

`HttpClient` هي الوحدة المدمجة في Angular التي توفر واجهة بسيطة لإجراء طلبات HTTP. قبل البدء، يجب استيراد هذه الوحدة في ملف `app.module.ts` لتصبح متاحة في التطبيق. لإرسال طلب جلب بيانات، نستخدم طريقة `.get()`.

مثال على جلب البيانات (GET Request)

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';

// لنوع البيانات المتوقع (Interface) تعريف واجهة
interface Post {
  id: number;
  title: string;
  body: string;
}

@Injectable({ providedIn: 'root' })
export class DataService {
  private apiUrl = 'https://jsonplaceholder.typicode.com/posts';
```



```

constructor(private http: HttpClient) {}

// دالة لجلب قائمة المنشورات من الخادم
getPosts(): Observable<Post[]> {
  return this.http.get<Post[]>(this.apiUrl);
}
}

```

شرح الكود

هذا الكود يوضح كيفية إنشاء خدمة (Service) تستخدم `HttpClient` لجلب البيانات. أولاً، نقوم بحقن خدمة `HttpClient` في البناء (constructor). ثم نستخدم طريقة `get()` لتنفيذ طلب HTTP GET إلى رابط API المحدد. لاحظ أننا حددنا نوع البيانات المتوقع (`Post[]`) بين علامتي `<` و `>` لضمان أمان النوع (type safety) في TypeScript.

- **العميل والخادم:** تطبيق Angular هو العميل، وهو يرسل طلبات للحصول على البيانات من الخادم.
- **HttpClient:** هو الأداة الرئيسية في Angular التي تسهل عملية إرسال هذه الطلبات واستقبال الردود.
- **الاستجابة كـ Observable:** طلبات `HttpClient` تعيد تدفق بيانات (Observable) يتطلب الاشتراك فيه (subscribe) للحصول على البيانات فعلياً ومعالجتها.

عميل (HTTP Client) HTTP — وحدة

`HttpClientModule` وإجراء طلبات GET

وحدة HttpClientModule وإجراء طلبات GET

في تطوير تطبيقات الويب الحديثة، من الضروري غالبًا جلب البيانات من خادم خارجي (API) لعرضها للمستخدم. في Angular، توفر وحدة `HttpClientModule` الأداة الرئيسية لإجراء طلبات HTTP بجميع أنواعها (مثل GET، POST، PUT، DELETE). هذه الوحدة هي أساس التواصل بين تطبيقك والخوادم الخارجية.

مفهوم أساسي: حقن HttpClient

لكي تتمكن من استخدام وظائف `HttpClientModule` لإرسال الطلبات، يجب عليك أولاً استيراد الوحدة في ملف `app.module.ts`. بعد ذلك، يجب حقن خدمة `HttpClient` في الباني (constructor) الخاص بالخدمة أو المكون (Component) الذي سيقوم بإجراء الطلب. طلب GET هو أبسط وأكثر أنواع الطلبات استخدامًا لجلب البيانات من الخادم.

مثال كود بسيط: جلب البيانات باستخدام GET

يوضح المثال التالي كيفية استخدام خدمة `HttpClient` داخل خدمة (Service) لجلب قائمة بالمستخدمين.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

// تعريف واجهة للبيانات المستلمة
interface User {
  id: number;
  name: string;
}

@Injectable({ providedIn: 'root' })
export class DataService {
```

```
private apiUrl = 'https://jsonplaceholder.typicode.com/users';

constructor(private http: HttpClient) {}

getUsers(): Observable<User[]> {
  return this.http.get<User[]>(this.apiUrl);
}
}
```

شرح الكود

في المثال أعلاه، نقوم أولاً بحقن خدمة `HttpClient` في الباني (constructor) لخدمة `DataService`. بعد ذلك، قمنا بإنشاء دالة `getUsers` التي تستخدم الدالة `get` من `HttpClient`. تحدد `` نوع البيانات المتوقعة، وتُرجع الدالة `Observable` يحتوي على مصفوفة من كائنات `User`.

- **أهمية HttpClientModule:** تمكن تطبيقك من التواصل مع واجهات برمجة التطبيقات (APIs) الخارجية.
- **حقن الخدمة:** يجب حقن `HttpClient` في الباني قبل استخدامها لإجراء أي طلب.
- **ال Observable:** طلب `get` يعيد `Observable`، مما يتطلب الاشتراك (subscribe) فيه لاستقبال البيانات الفعلية.

عميل (HTTP Client) HTTP — إجراء طلبات POST, PUT, DELETE

إجراء طلبات POST, PUT, DELETE في Angular

في تطوير التطبيقات، نحتاج غالبًا إلى التفاعل مع الخادم لإدارة البيانات، وهي عملية تُعرف بالاختصار CRUD (إنشاء، قراءة، تحديث، حذف). يوفر [Angular HttpClient](#) أساليب مخصصة لتنفيذ طلبات HTTP التي تغير حالة البيانات على الخادم.

مفهوم طلبات HTTP لتعديل البيانات

تُستخدم هذه الطلبات لإجراء تغييرات على الموارد الموجودة على الخادم:

- **POST:** يُستخدم لإنشاء مورد جديد. نقوم بإرسال البيانات (Data Body) إلى الخادم لإنشاء سجل جديد.
- **PUT:** يُستخدم لتحديث مورد موجود بالكامل. يتطلب تحديد المورد المراد تحديثه (غالبًا عبر معرفه ID) وإرسال البيانات الجديدة.
- **DELETE:** يُستخدم لحذف مورد موجود. يتطلب تحديد المورد المراد حذفه عن طريق معرفه (ID) في مسار URL.

مثال على استخدام POST, PUT, DELETE في خدمة Angular

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  private apiUrl = 'https://api.example.com/products';

  constructor(private http: HttpClient) {}
```

```
// POST: إنشاء منتج جديد
createProduct(productData: any): Observable<any> {
  return this.http.post(this.apiUrl, productData);
}

// PUT: تحديث منتج موجود
updateProduct(productId: number, newProductData: any): Observable<any> {
  return this.http.put(`${this.apiUrl}/${productId}`, newProductData);
}

// DELETE: حذف منتج
deleteProduct(productId: number): Observable<any> {
  return this.http.delete(`${this.apiUrl}/${productId}`);
}
}
```

شرح الكود

يوضح هذا المثال ثلاثة أساليب شائعة لإدارة البيانات باستخدام خدمة Angular:

- **createProduct(productData)**: يستخدم أسلوب **post()** لإرسال كائن **productData** إلى المسار الأساسي (**this.apiUrl**) لإنشاء سجل جديد.
- **updateProduct(productId, newProductData)**: يستخدم أسلوب **put()**، حيث يتم تضمين **productId** في مسار URL لتحديد المورد المراد تحديثه، وإرسال البيانات الجديدة **newProductData** في جسم الطلب.
- **deleteProduct(productId)**: يستخدم أسلوب **delete()**، حيث يتم تضمين **productId** في مسار URL لإخبار الخادم بالمنتج المحدد الذي يجب إزالته.

عميل (HTTP Client) HTTP — التعامل مع الاستجابات والأخطاء (Error Handling)

التعامل مع الاستجابات والأخطاء (Error Handling)

عند إجراء طلبات HTTP، لا نتوقع النجاح دائماً. قد تحدث أخطاء بسبب مشاكل في الشبكة، أو أخطاء في الخادم (مثل 404 Not Found أو 500 Server Error). التعامل الصحيح مع هذه الأخطاء أمر حاسم لضمان استقرار التطبيق وتقديم تجربة مستخدم جيدة. في Angular، يتم ذلك باستخدام مفهوم الـ Observables.

مفهوم معالجة الأخطاء في Angular

عندما نستخدم `HttpClient` لإرسال طلب، فإنه يعيد `Observable`. للاستماع إلى النتيجة، نستخدم دالة `subscribe()`. تقبل هذه الدالة كائناً يحتوي على معالجين (callbacks) رئيسيين: `next` للتعامل مع الاستجابة الناجحة، و `error` للتعامل مع الأخطاء.

مثال كود بسيط

```
import { HttpClient } from '@angular/common/http';

export class DataService {

  constructor(private http: HttpClient) {}

  fetchData() {
    this.http.get('/api/users').subscribe({
      next: (response) => {
        // يتم تنفيذ هذا الجزء عند نجاح الطلب (status 200)
        console.log('Success response:', response);
      },
      error: (err) => {
        // يتم تنفيذ هذا الجزء عند فشل الطلب (status 4xx, 5xx)
        console.error('An error occurred:', err);
        // عرض رسالة خطأ للمستخدم
      }
    });
  }
}
```

```
});  
}  
}
```

شرح الكود

في هذا المثال، نستخدم `subscribe()` للاستماع إلى نتيجة طلب `GET`. إذا نجح الطلب واستقبلنا بيانات (رمز حالة 200)، يتم استدعاء الدالة `next`. أما إذا حدث خطأ (مثل خطأ في الخادم أو عدم العثور على المورد)، يتم استدعاء الدالة `error`. هذا يتيح لنا فصل منطق النجاح عن منطق الفشل.

- **ال Observable:** يعيد `HttpClient` كائن `Observable` يجب "الاشتراك" به للاستماع للبيانات.
- **معالج النجاح (next):** الدالة التي تُنفذ عند استقبال بيانات صحيحة من الخادم.
- **معالج الخطأ (error):** الدالة التي تُنفذ عند فشل الطلب، وتسمح لك بالتعامل مع المشكلة.

عميل (HTTP Client) HTTP — المراقبة

(Observables) في سياق HTTP

المراقبة (Observables) في سياق HTTP

في إطار تطوير تطبيقات Angular، تُعد المراقبة (Observables) جزءًا أساسيًا للتعامل مع العمليات غير المتزامنة، خاصةً عند إجراء طلبات HTTP لجلب البيانات من خوادم خارجية. بدلاً من إعادة البيانات بشكل مباشر، يقوم عميل Angular HTTP بإرجاع "مراقبة" (Observable) يمثل تيارًا من البيانات سيصل في المستقبل.

مفهوم المراقبة (Observable)

يمكن التفكير في المراقبة على أنه تيار بيانات قابل للاشتراك. عند إجراء طلب HTTP، لا يتم تنفيذه فورًا. بدلاً من ذلك، تحصل على مراقبة. لتفعيل الطلب والحصول على البيانات الفعلية عند وصولها، يجب عليك "الاشتراك" (subscribe) في هذا المراقبة. عند اكتمال الطلب بنجاح، يقوم المراقبة بتمرير البيانات إلى المشترك (Subscriber).

مثال الكود (Angular Service)

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  constructor(private http: HttpClient) {}

  getUsers(): Observable<any[]> {
    return this.http.get<any[]>('https://api.example.com/users');
  }
}
```



```
// داخل المكون (Component)
this.dataService.getUsers().subscribe(
  (data) => {
    // معالجة البيانات التي تم استلامها
    console.log('Received data:', data);
  },
  (error) => {
    // معالجة الأخطاء
    console.error('An error occurred:', error);
  }
);
```

شرح الكود

يقوم الكود أعلاه بتوضيح كيفية استخدام المرتقبات. في الدالة `getUsers()`، يتم إرجاع مرتقب (Observable). هذا المرتقب لا يقوم بتنفيذ طلب HTTP فعليًا حتى يتم استدعاء الدالة `subscribe()`. عند الاشتراك، يتم إرسال الطلب، وعندما يعود الخادم بالبيانات، يتم تنفيذ وظيفة الـ callback الأولى `((data) => ...)` لمعالجة البيانات المستلمة، بينما يتم تنفيذ وظيفة الـ callback الثانية `((error) => ...)` في حال حدوث خطأ.

- **غير متزامن (Asynchronous):** تسمح المرتقبات بالتعامل مع البيانات التي ستصل في وقت لاحق، دون حجب تنفيذ باقي الكود.
- **الكسل (Lazy):** لا يتم تنفيذ طلب HTTP حتى يتم الاشتراك في المرتقب (Observable) العائد منه.
- **الاشتراك (Subscription):** هي الآلية التي نبدأ بها تنفيذ الطلب ونستمع لنتائجه (سواء كانت بيانات ناجحة أو خطأ).

المرتقبات (Observables) و RxJS — مقدمة إلى

RxJS و ReactiveX

html

مقدمة إلى RxJS و ReactiveX

في تطوير التطبيقات الحديثة، نحتاج إلى التعامل مع البيانات المتغيرة باستمرار والمصادر غير المتزامنة، مثل طلبات واجهة برمجة التطبيقات (API) أو تفاعلات المستخدم. RxJS هي مكتبة قوية تتيح لنا إدارة "تيارات البيانات" (data streams) هذه بأسلوب تفاعلي (reactive) وبشكل منظم، مما يجعلها أساسية في إطار عمل Angular.

مفهوم البرمجة التفاعلية (Reactive Programming)

البرمجة التفاعلية هي نموذج برمجي يتعامل مع تيارات البيانات المتدفقة بمرور الوقت. بدلاً من طلب البيانات عند الحاجة (نموذج السحب)، فإننا "نشارك" في مصدر البيانات لتتلقى تحديثات عندما تصبح متاحة (نموذج الدفع). المرتقب (Observable) هو اللبنة الأساسية في هذا النموذج، حيث يمثل تيار البيانات الذي يمكن للمشتركين الاستماع إليه.

مثال: إنشاء مرتقب بسيط باستخدام RxJS

```
import { of } from 'rxjs';

// إصدار الأرقام 10، 20، 30 (Observable) إنشاء مرتقب
const numbers$ = of(10, 20, 30);

// الاشتراك في المرتقب لبدء استقبال البيانات
numbers$.subscribe({
  next: (value) => console.log(`Received value: ${value}`),
  complete: () => console.log('Observable completed!')
});
```

شرح الكود

في هذا المثال، نستخدم الدالة `of` لإنشاء مرتقب يبعث القيم (10، 20، 30) على التوالي. الدالة `subscribe` هي التي تبدأ الاستماع إلى هذا التيار. عند الاشتراك، يتم استدعاء `next` لكل قيمة يتم إصدارها، وعند الانتهاء من جميع القيم، يتم استدعاء `complete` للإشارة إلى اكتمال التيار.

- **المرتقب (Observable):** هو مصدر البيانات.
- **الاشتراك (Subscription):** هو المستمع للبيانات، وهو ما يبدأ تشغيل المرتقب.
- **البرمجة التفاعلية (Reactive Programming):** تساعدنا على بناء تطبيقات تستجيب للتغيرات في البيانات بشكل تلقائي.

المرتقبات (Observables) و RxJS — مفهوم

المرتقبات (Observables) والمراقبين (Observers)

مفهوم المرتقبات (Observables) والمراقبين (Observers)

تُعد المرتقبات (Observables) من المفاهيم الأساسية في Angular لإدارة البيانات غير المتزامنة (Asynchronous data). إنها طريقة للتعامل مع تدفق البيانات التي قد تصل في المستقبل، سواء كانت استجابة من خادم (API call) أو أحداث المستخدم (مثل النقر على زر).

المراقب (Observable) والمراقب (Observer)

يمكنك تخيل المرتقب كـ "مصدر بيانات" أو "ناشر" (Publisher) يقوم بإرسال سلسلة من الأحداث أو القيم بمرور الوقت. المراقب هو "مستهلك البيانات" أو "المشارك" (Subscriber) الذي ينتظر لاستقبال هذه القيم. العلاقة بينهما تتم من خلال عملية "الاشتراك" (Subscription).

مثال برمجي

```
import { Observable } from 'rxjs';

// إنشاء المرتقب (مصدر البيانات) 1.
const sourceObservable = new Observable(observer => {
  observer.next('قيمة 1'); // إرسال القيمة الأولى
  observer.next('قيمة 2'); // إرسال القيمة الثانية
  observer.complete(); // إتمام التدفق
});

// إنشاء المراقب (مستهلك البيانات) 2.
const dataObserver = {
  next: (value: string) => console.log(`استلمت: ${value}`),
  error: (err: any) => console.error(err),
  complete: () => console.log('تم اكتمال التدفق')
};
```

```
// الاشتراك لربط المراقب بالمرتقب 3.  
sourceObservable.subscribe(dataObserver);
```

شرح الكود

في هذا المثال، قمنا بإنشاء مرتقب (`sourceObservable`) يصدر قيمتين ثم يعلن عن اكتمال التدفق. بعد ذلك، قمنا بتعريف المراقب (`dataObserver`) الذي يحتوي على ثلاث دوال: `next` لاستلام البيانات، `error` للتعامل مع الأخطاء، و `complete` للإشارة إلى نهاية التدفق. عندما يتم استدعاء `subscribe()`، يبدأ المراقب في تلقي القيم من المرتقب.

- **المرتقب (`Observable`):** مصدر البيانات الذي يمكن أن يصدر قيمًا متعددة بمرور الوقت.
- **المراقب (`Observer`):** كائن يحتوي على دوال (`next`, `error`, `complete`) للتعامل مع البيانات المستلمة.
- **الاشتراك (`Subscription`):** العلاقة بين المرتقب والمراقب التي تتيح للمراقب بدء استقبال البيانات.

المرتقبات (Observables) و RxJS — الاشتراك في المرتقبات (Subscribing) وإلغاء الاشتراك

html

الاشتراك في المرتقبات (Subscribing) وإلغاء الاشتراك

في عالم Angular و RxJS، لا تبدأ المرتقبات (Observables) في إرسال البيانات من تلقاء نفسها. إنها مثل قناة تلفزيونية مغلقة لا يتم بثها إلا عندما يقوم المشاهد بـ "الاشتراك" فيها. عملية الاشتراك هذه هي التي تطلق المرتقب وتسمح لك بالوصول إلى البيانات التي يقدمها. يعد فهم كيفية الاشتراك وإلغاء الاشتراك أمرًا أساسيًا لضمان عمل تطبيقك بكفاءة.

مفهوم الاشتراك (Subscribe)

الاشتراك هو الخطوة التي تطلق المرتقب لبدء إرسال البيانات. يتم ذلك عن طريق استدعاء الدالة `subscribe()`. تقبل هذه الدالة عادةً دالة لمعالجة البيانات القادمة (`next`)، ودالة لمعالجة الأخطاء (`error`)، ودالة للاكتمال (`complete`).

مثال على الاشتراك وإلغاء الاشتراك

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { Observable, Subscription } from 'rxjs';

@Component({
  selector: 'app-data',
  template: '{{ data }}'
})
export class DataComponent implements OnInit, OnDestroy {
  data: string = 'Loading...';
  private dataSubscription: Subscription;

  ngOnInit(): void {
    const dataObservable = new Observable(observer => {
      setTimeout(() => observer.next('Data loaded!'), 2000);
    });
```

```

});

this.dataSubscription = dataObservable.subscribe(
  value => this.data = value,
  error => console.error(error)
);
}

ngOnDestroy(): void {
  if (this.dataSubscription) {
    this.dataSubscription.unsubscribe();
  }
}
}

```

شرح الكود

في هذا المثال، قمنا بإنشاء مرقب بسيط. يتم الاشتراك فيه داخل دورة حياة `ngOnInit` باستخدام `subscribe()`، مما يطلق المرقب ويحدث قيمة المتغير `data`. عند تدمير المكون (على سبيل المثال، عند الانتقال إلى صفحة أخرى)، يتم استدعاء `ngOnDestroy`، حيث نستخدم `unsubscribe()` لإيقاف تدفق البيانات. هذا الإجراء ضروري لمنع تسرب الذاكرة (Memory Leaks).

- **الاشتراك (Subscribe):** هو العملية التي تبدأ في استلام البيانات من المرقب.
- **إلغاء الاشتراك (Unsubscribe):** هو إنهاء عملية الاستماع للمرقب وتحرير الموارد.
- **تجنب تسرب الذاكرة:** يجب دائماً إلغاء الاشتراك في المكونات (Components) داخل دورة حياة `ngOnDestroy`، خاصة للمرتهبات طويلة الأمد.

المرتقبات (RxJS Observables) — العوامل

الشائعة (Operators): `map`, `filter`, `debounceTime`

العوامل الشائعة (Operators): `map`, `filter`, `debounceTime`

في عالم Angular و RxJS، المرتقبات (Observables) هي تيارات من البيانات. العوامل (Operators) هي أدوات قوية نستخدمها لتعديل أو تصفية هذه البيانات قبل أن تصل إلى المشترك (Subscriber). هذه العوامل تتيح لنا التحكم الكامل في تدفق البيانات وتحويلها حسب الحاجة، مما يجعل التعامل مع الأحداث غير المتزامنة أكثر فعالية.

مفهوم العوامل الأساسية

تعمل العوامل كمرشحات ووظائف تحويل على البيانات الصادرة من المرتقب. سنتعرف على ثلاثة عوامل رئيسية تستخدم بكثرة في بناء التطبيقات:

- **`map`**: لتحويل البيانات.
- **`filter`**: لتصفية البيانات.
- **`debounceTime`**: للتحكم في توقيت إرسال البيانات.

مثال برمجي

سنستخدم مثالاً شائعاً حيث نرصد ضغط المفاتيح في حقل إدخال (input) ونطبق عليها العوامل.

```
import { fromEvent } from 'rxjs';
import { map, filter, debounceTime } from 'rxjs/operators';

// 'searchInput' نفترض أن لدينا حقل إدخال باسم
const searchInput = document.getElementById('searchInput');

fromEvent(searchInput, 'keyup').pipe(
  map((event: any) => event.target.value),
```



```
filter((text: string) => text.length > 3),
debounceTime(500)
).subscribe(searchTerm => {
  console.log(`Searching for: ${searchTerm}`);
});
```

شرح الكود

يبدأ الكود بمراقبة حدث `keyup` (رفع المفتاح) في حقل الإدخال. يقوم عامل `map` بتحويل حدث المفتاح إلى قيمة النص الموجودة في الحقل. بعد ذلك، يقوم عامل `filter` بتجاهل أي قيمة يكون طولها أقل من 4 أحرف. العامل الأخير، `debounceTime(500)`، هو الأهم هنا: فبدلاً من إرسال القيمة مع كل ضغطة مفتاح، ينتظر 500 مللي ثانية (نصف ثانية) بعد آخر ضغطة. إذا استمر المستخدم في الكتابة، يتم تأجيل الإرسال، مما يقلل عدد الاستدعاءات غير الضرورية لعملية البحث.

ملخص الفكرة

- **`map`**: يُستخدم لتحويل شكل البيانات القادمة من المرتقب (كاستخراج قيمة نصية من حدث).
- **`filter`**: يُستخدم لتصفية البيانات بناءً على شروط منطقية محددة قبل معالجتها.
- **`debounceTime`**: يُستخدم لتأخير إرسال البيانات وتقليل التحميل على النظام، خاصة في حقول البحث، عن طريق انتظار فترة توقف محددة.

المرتقبات (Observables) و RxJS — العوامل

الشائعة: `switchMap`، `tap`

html

العوامل الشائعة: `switchMap` و `tap`

RxJS يمنحنا أدوات قوية للتحكم في تدفق البيانات. سنتناول عاملين شائعين جدًا في تطبيقات Angular: **switchMap** و **tap**. الأول يستخدم لإدارة الطلبات المتداخلة (Nested Observables)، والثاني لإضافة تأثيرات جانبية (Side Effects) دون تغيير البيانات المارة.

مفهوم `switchMap`

يُستخدم `switchMap` عندما يكون لديك مرتقب (Observable) يعتمد على قيمة من مرتقب آخر. وظيفته الأساسية هي إلغاء المرتقب الداخلي السابق إذا صدرت قيمة جديدة من المرتقب الخارجي. هذا يمنع ما يسمى "ظاهرة سباق البيانات" (Race Condition) ويضمن أنك تتعامل دائمًا مع أحدث البيانات.

مفهوم `tap`

يسمح لك عامل `tap` بتنفيذ كود برمجي كـ "تأثير جانبي" أثناء مرور البيانات عبر الأنبوب (pipeline) دون تغيير قيمة البيانات نفسها. إنه مفيد جدًا لتتبع البيانات أو تسجيلها في السجل (Console) أو تحديث حالة واجهة المستخدم قبل استكمال المعالجة.

مثال عملي

```
// يبحث بقيمة حقل البحث Observable هو searchTerms نفترض أن
searchTerms.pipe(
  // لتسجيل قيمة البحث دون تغييرها tap نستخدم
  tap(term => console.log(`Searching for: ${term}`)),
  // لتبديل المرتقب الخارجي بطلب switchMap نستخدم
  switchMap(term => this.dataService.searchUsers(term))
)
```

```
).subscribe(results => {  
  // عرض النتائج  
  this.searchResults = results;  
});
```

شرح الكود

عندما تتغير قيمة حقل البحث، يبدأ الأنبوب. يقوم `tap` بتسجيل قيمة البحث في سجل المتصفح (Console) للمراقبة. ثم يستخدم `switchMap` هذه القيمة لطلب البيانات من الخدمة. الأهم هنا هو أن `switchMap` يلغي أي طلب بحث سابق إذا لم يكن قد اكتمل بعد، مما يضمن أننا نعرض نتائج البحث الأحدث فقط.

- **switchMap**: يلغي الاشتراكات الداخلية السابقة ويضمن دائمًا التعامل مع أحدث البيانات.
- **tap**: يسمح لك بمراقبة المرتقب وتنفيذ تأثيرات جانبية دون تعديل البيانات.
- استخدامهما معًا شائع في سيناريوهات البحث التلقائي (Autocomplete) حيث يتم إلغاء طلبات البحث القديمة بسرعة.

~~~

# الأنابيب (Pipes) — مفهوم الأنابيب ودورها في تحويل البيانات

## مفهوم الأنابيب ودورها في تحويل البيانات

تُعد الأنابيب (Pipes) من الميزات الأساسية في Angular التي تساعد في إدارة عرض البيانات. إنها تسمح لك بتغيير طريقة ظهور البيانات في قالب HTML دون الحاجة لتغيير البيانات الأصلية في ملف المكون (component). الأنبوب ببساطة يأخذ قيمة كمدخل، ويجري عليها عملية تحويل، ثم يعرض القيمة المحوّلة.

### ما هي الأنابيب (Pipes)؟

الأنابيب هي وظائف تستخدم لتنسيق البيانات مباشرة في القالب. تخيل أن لديك تاريخًا وتريد عرضه بتنسيق معين (مثل "15 يوليو 2025")، أو لديك رقمًا وتريد تنسيقه كعملة (مثل "\$100.00"). بدلاً من كتابة كود معقد لتحويل هذه البيانات، يمكنك استخدام الأنابيب الجاهزة التي يوفرها Angular.

### مثال على استخدام الأنبوب (Pipe)

```
// component.ts
export class MyComponent {
  today: Date = new Date();
  price: number = 1234.56;
}

// HTML (component.html)
<!-- عرض التاريخ الأصلي -->
<p>التاريخ: {{ today }}</p>

<!-- استخدام الأنبوب لتنسيق التاريخ -->
<p>التاريخ المنسق: {{ today | date: 'mediumDate' }}</p>

<!-- استخدام الأنبوب لتنسيق السعر كعملة -->
```

```
<p>{{ price | currency: 'USD' }}</p>
```

## شرح الكود

في هذا المثال، نستخدم الأنابيب لتغيير طريقة عرض المتغيرين `today` و `price`. يتم تمرير البيانات إلى الأنبوب باستخدام الرمز `|`. يأخذ الأنبوب `date` القيمة `today` ويقوم بتنسيقها حسب النمط `'mediumDate'`. وبالمثل، يأخذ الأنبوب `currency` القيمة `price` ويحولها إلى تنسيق العملة الدولارية `'USD'`.

- **تحويل البيانات في القالب:** الأنابيب تسمح بتعديل عرض البيانات دون تغيير البيانات المصدر.
- **سهولة الاستخدام:** يتم تطبيق الأنبوب باستخدام رمز الشرطة العمودية `|` متبوعًا باسم الأنبوب.
- **أنابيب جاهزة:** يوفر Angular مجموعة كبيرة من الأنابيب الجاهزة مثل `date` و `currency` و `uppercase` و `lowercase`.

# الأنابيب (Pipes) — الأنابيب المدمجة (Built-in)

## (Pipes): `DatePipe`, `CurrencyPipe`

html

## الأنابيب المدمجة: `DatePipe` و `CurrencyPipe`

تُعد الأنابيب (Pipes) إحدى الميزات الأساسية في Angular التي تسمح بتحويل البيانات وعرضها بشكل منسق في القوالب (templates) دون الحاجة لتعديل البيانات الأصلية في المكون. توفر Angular مجموعة من الأنابيب المدمجة (Built-in Pipes) الجاهزة للاستخدام، أشهرها `DatePipe` لتنسيق التواريخ و `CurrencyPipe` لتنسيق العملات.

## مفهوم الأنابيب المدمجة

الأنابيب المدمجة هي وظائف جاهزة تستخدم الرمز `|` لتطبيق تنسيق معين على متغير في القالب. يعمل `DatePipe` على تحويل كائن التاريخ إلى تنسيق نصي سهل القراءة، بينما يقوم `CurrencyPipe` بتحويل رقم إلى تنسيق عملة محدد، مع إضافة الرمز المناسب وفواصل الآلاف.

## مثال تطبيقي (Template Code)

```
<!-- TypeScript: -->
<!-- export class MyComponent { -->
<!--   purchaseDate = new Date(); -->
<!--   productPrice = 1234.56; -->
<!-- } -->

<!-- استخدام DatePipe -->
<h3>تنسيق التاريخ</h3>
<p>التنسيق القصير: {{ purchaseDate | date:'shortDate' }}</p>
<p>التنسيق المخصص: {{ purchaseDate | date:'dd/MM/yyyy' }}</p>

<!-- استخدام CurrencyPipe -->
<h3>تنسيق العملة</h3>
```

```
<p>{{ productPrice | currency:'USD' }}</p>
<p>{{ productPrice | currency:'EUR': 'symbol':'1.0-0' }}</p>
```

## شرح الكود

في المثال أعلاه، يتم استخدام الأنابيب لتحويل البيانات. في السطر الأول، يقوم `DatePipe` بتطبيق تنسيق `'shortDate'` (تنسيق تاريخ مدمج وقصير) على المتغير `purchaseDate`. في السطر الثاني، يتم تمرير تنسيق مخصص `'dd/MM/yyyy'` لتحقيق تنسيق معين. بالنسبة للعملة، يقوم `CurrencyPipe` بتنسيق `productPrice` إلى عملة الدولار الأمريكي (USD). في مثال اليورو (EUR)، يتم استخدام معلمات إضافية (مثل `'symbol'`) لتحديد طريقة عرض الرمز وعدد الخانات العشرية.

- **تحويل البيانات في القوالب:** تستخدم الأنابيب الرمز `|` لتحويل البيانات مباشرة في HTML.
- **سهولة الاستخدام:** الأنابيب المدمجة مثل `DatePipe` و `CurrencyPipe` جاهزة للاستخدام الفوري دون استيرادها بشكل صريح.
- **تخصيص التنسيق:** يمكن تمرير معلمات (Arguments) إلى الأنابيب بعد علامة النقطتين `:` لتخصيص التنسيق المطلوب.

...

# الأنابيب (Pipes) — الأنابيب المدمجة: `JsonPipe`, `SlicePipe`, `AsyncPipe`

## الأنابيب المدمجة: `JsonPipe`, `SlicePipe`, `AsyncPipe`

الأنابيب المدمجة (Built-in Pipes) هي مجموعة من الأدوات الجاهزة التي يوفرها Angular لتسهيل تحويل البيانات وعرضها داخل قوالب HTML مباشرة. تسمح هذه الأنابيب بتعديل شكل البيانات قبل عرضها للمستخدم دون الحاجة لكتابة منطق معقد في ملف المكون (Component).

### مفهوم الأنابيب المدمجة

تُستخدم الأنابيب المدمجة لتغيير تنسيق البيانات أو اقتطاعها أو التعامل مع تدفقات البيانات غير المتزامنة (Asynchronous data). من أشهر هذه الأنابيب:

- **`JsonPipe`**: يحول كائن JavaScript إلى سلسلة نصية بصيغة JSON. مفيد جدًا لعرض محتوى الكائن بالكامل لأغراض التصحيح.
- **`SlicePipe`**: يسمح لك باقتطاع جزء من سلسلة نصية أو مصفوفة بناءً على فهرس البداية وفهرس النهاية.
- **`AsyncPipe`**: يتعامل مع البيانات غير المتزامنة (Observable أو Promise)، حيث يقوم بالاشتراك (Subscribing) في البيانات وعرضها، ثم إلغاء الاشتراك تلقائيًا عند انتهاء المكون، مما يمنع تسرب الذاكرة.

### مثال تطبيقي

افترض أن لديك البيانات التالية في ملف المكون (TS):

```
import { Component } from '@angular/core';
import { Observable, of } from 'rxjs';

@Component({
  selector: 'app-example',
  template: `
    <h3>بيانات الكائن (JsonPipe)</h3>`
})
```



```

<pre>{{ user | json }}</pre>

<h3>اقتطاع مصفوفة (SlicePipe)</h3>
<p>المصفوفة الأصلية: {{ numbers }}</p>
<p>المصفوفة المقتطعة: {{ numbers | slice:1:4 }}</p>

<h3>بيانات غير متزامنة (AsyncPipe)</h3>
<p>{{ message$ | async }}</p>
,
}))
export class ExampleComponent {
  user = { name: 'Ahmad', role: 'Developer' };
  numbers = [1, 2, 3, 4, 5];
  message$: Observable<string> = of('Hello from Angular');
}

```

## شرح الكود

يوضح المثال كيفية استخدام الأنابيب الثلاثة مباشرة في قالب HTML:

- يستخدم `user | json` لتحويل كائن المستخدم إلى نص JSON قابل للعرض.
- يستخدم `numbers | slice:1:4` لاقتطاع العناصر من الفهرس 1 حتى الفهرس 4 (أي الأرقام 2، 3، 4) من المصفوفة.
- يستخدم `message$ | async` للاشتراك تلقائيًا في تدفق البيانات ``message`` وعرض قيمته "Hello from Angular" بمجرد توفرها.

## ملخص الفكرة

- الأنابيب المدمجة تسهل تنسيق البيانات وعرضها مباشرة في القوالب.
- يُستخدم رمز `|` لتطبيق الأنابيب، ويمكن تمرير المعاملات باستخدام رمز النقطتين `:`.
- يُعد `AsyncPipe` ضروريًا للتعامل الآمن مع البيانات غير المتزامنة (Observables) ومنع تسرب الذاكرة في Angular.

# الأنابيب (Pipes) — استخدام الأنابيب في القوالب

## استخدام الأنابيب في القوالب (Using Pipes in Templates)

الأنابيب (Pipes) هي واحدة من الميزات القوية في Angular التي تسمح لك بتحويل وعرض البيانات داخل قوالب HTML بطريقة سهلة وفعالة. بدلاً من كتابة دوال معقدة في ملف TypeScript، يمكنك استخدام الأنبوب لتنسيق البيانات (مثل التواريخ والعملات والنصوص) مباشرة في القالب.

### مفهوم الأنبوب الأساسي

يتم تطبيق الأنبوب باستخدام الرمز `|` (يُسمى "pipe operator"). هذا الرمز يأخذ القيمة الموجودة على يساره ويمررها إلى الأنبوب الموجود على يمينه لتحويلها. النتيجة النهائية للتحويل هي ما يتم عرضه على الشاشة.

### مثال على استخدام الأنابيب

```
<!-- ومتغير currentDate = new  
username = "john doe" افترض أن لدينا متغير --!>  
<div>  
  <p>التنسيق: {{ username }}</p>  
  <p>التنسيق (uppercase pipe): {{ username | uppercase }}</p>  
  <p>التاريخ الافتراضي: {{ currentDate | date }}</p>  
  <p>التاريخ بصيغة مختصرة (shortDate pipe): {{ currentDate | date:'shortDate' }}</p>  
</div>
```

### شرح الكود

في المثال أعلاه، يتم استخدام الأنابيب لتحويل البيانات. في السطر الأول، يُعرض الاسم بحالته الأصلية. في السطر الثاني، يتم تمرير متغير `username` عبر الأنبوب `uppercase`، مما يحول جميع الحروف إلى أحرف كبيرة قبل عرضها.

أما بالنسبة للتاريخ، فيتم استخدام الأنبوب `date` في المرة الأولى، يُعرض التاريخ بالتنسيق الافتراضي. في المرة الثانية، يتم تمرير وسيط (argument) وهو `'shortDate'` إلى الأنبوب، ليتم عرض التاريخ بتنسيق مختصر محدد مسبقًا.

- **بساطة التنسيق:** توفر الأنابيب طريقة نظيفة لتنسيق البيانات دون الحاجة إلى منطق إضافي في ملف المكون (component file).
- **سلسلة الأنابيب (Chaining):** يمكن ربط عدة أنابيب معًا (piping chains) لتطبيق تحويلات متعددة على نفس البيانات (مثال: `{{ data | pipe1 | pipe2 }}`).
- **الأنابيب المدمجة والمخصصة:** يوفر Angular مجموعة كبيرة من الأنابيب الجاهزة للاستخدام (مثل `date`، `currency`، `json`)، كما يمكنك إنشاء أنابيبك المخصصة.

# الأنابيب (Pipes) — إنشاء أنابيب مخصصة (Custom Pipes)

## إنشاء أنابيب مخصصة (Custom Pipes)

تُستخدم الأنابيب (Pipes) في Angular لتحويل البيانات قبل عرضها في القالب (Template). بينما توفر Angular مجموعة من الأنابيب الجاهزة (Built-in Pipes)، نحتاج في بعض الأحيان إلى إنشاء أنابيب خاصة بنا لتلبية متطلبات فريدة في التطبيق. تسمى هذه الأنابيب بالأنابيب المخصصة (Custom Pipes).

### مفهوم الأنابيب المخصصة

تسمح الأنابيب المخصصة بإنشاء تحويلات بيانات قابلة لإعادة الاستخدام. لإنشاء أنبوب مخصص، نستخدم المزخرف `Pipe@` لتحديد اسم الأنبوب، ويجب أن يحتوي الكلاس على الدالة `transform` التي تنفذ عملية التحويل. هذه الدالة تستقبل قيمة الإدخال (input value) وأي معاملات إضافية (optional arguments) وتعيد القيمة المحولة.

### مثال على أنبوب مخصص

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'reverseText'
})
export class ReverseTextPipe implements PipeTransform {
  transform(value: string): string {
    return value.split('').reverse().join('');
  }
}
```

### شرح الكود

يقوم هذا المثال بإنشاء أنبوب مخصص باسم `reverseText`. يتم استخدام المزخرف `Pipe@` لتسجيل الأنبوب. الكلاس `ReverseTextPipe` يقوم بتطبيق الواجهة `PipeTransform` التي تتطلب وجود الدالة `transform`. هذه الدالة تستقبل نصًا (value) وتقوم بعكس ترتيب أحرفه ثم تعيد النص معكوسًا.

- **إعادة الاستخدام:** تتيح الأنابيب المخصصة توحيد منطق التحويل وإعادة استخدامه في أماكن مختلفة من التطبيق.
- **فصل الاهتمامات:** تساعد الأنابيب على فصل منطق العرض عن منطق المكونات لزيادة وضوح الكود.
- **التطبيق:** يتم استخدام الأنابيب المخصصة في القوالب (Templates) بنفس طريقة استخدام الأنابيب الجاهزة: `{{ data | pipeName }}`.

# إدارة الحالة الأساسية (Basic State Management)

## — مقدمة لمفهوم إدارة الحالة في تطبيقات الويب

### مقدمة لمفهوم إدارة الحالة في تطبيقات الويب

في بناء تطبيقات الويب المعقدة، تحتاج المكونات المختلفة إلى الوصول إلى نفس البيانات المشتركة وتحديثها. على سبيل المثال، بيانات المستخدم المسجل دخوله أو عربة التسوق. هذه البيانات المتغيرة تُعرف باسم "الحالة" (State). إدارة الحالة هي عملية تنظيم هذه البيانات بحيث يسهل تتبعها وتحديثها عبر التطبيق بأكمله.

### ما هي إدارة الحالة؟

إدارة الحالة هي أسلوب برمجي يهدف إلى تسهيل مشاركة البيانات بين مكونات التطبيق. بدلاً من تمرير البيانات من مكون أب إلى مكون ابن، يتم وضع البيانات في "مكان مركزي" يمكن لجميع المكونات الوصول إليه بشكل مباشر. هذا يقلل من تعقيد التطبيق ويجعل صيانتها أسهل، خاصة عند نمو التطبيق.

### مشاركة الحالة الأساسية في Angular باستخدام الخدمات (Services)

في Angular، تُعد الخدمات (Services) هي الطريقة القياسية لإدارة الحالة الأساسية. يمكن للمكونات المختلفة حقن نفس الخدمة، وبذلك تشترك في نفس نسخة البيانات المخزنة داخل هذه الخدمة. نستخدم عادةً مكتبة RxJS (التي يتم تضمينها مع Angular) لإنشاء تدفقات بيانات تفاعلية (reactive data streams) تسمح للمكونات بالاستماع إلى التحديثات فور حدوثها.

### مثال: خدمة حالة بسيطة (Counter Service)

```
// counter.service.ts
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
```

```

export class CounterService {
  private countSubject: BehaviorSubject<number> = new BehaviorSubject<number>
  public count$: Observable<number> = this.countSubject.asObservable();

  constructor() {}

  incrementCounter() {
    this.countSubject.next(this.countSubject.value + 1);
  }

  resetCounter() {
    this.countSubject.next(0);
  }
}

```

## شرح الكود

في هذا المثال، قمنا بإنشاء خدمة تُسمى `CounterService`. تحتوي هذه الخدمة على `BehaviorSubject` يُسمى `countSubject` يحتفظ بالقيمة الحالية للعداد. عندما يتم استدعاء دالة `incrementCounter()`، يتم تحديث القيمة في `BehaviorSubject`. يمكن لأي مكون في التطبيق أن يشترك في `$count` (وهو نسخة عامة من `countSubject`) ليقوم بتحديث واجهة المستخدم تلقائيًا عند تغيير القيمة.

- **ما هي الحالة؟** هي البيانات المتغيرة التي تُستخدم لعرض واجهة المستخدم، مثل بيانات المستخدم أو نتائج البحث.
- **لماذا نستخدم إدارة الحالة؟** لتسهيل مشاركة البيانات بين مكونات التطبيق المختلفة وتجنب تعقيد تمرير البيانات يدويًا.
- **كيف تتم الإدارة الأساسية في Angular؟** باستخدام الخدمات (Services) لمركزية البيانات و RxJS لتحديث المكونات بشكل تفاعلي.

# إدارة الحالة الأساسية (Basic State Management)

## — مشاركة البيانات بين المكونات باستخدام الخدمات و`BehaviorSubject`

html

### إدارة الحالة الأساسية: مشاركة البيانات بين المكونات باستخدام الخدمات و`BehaviorSubject`

في تطبيقات Angular المعقدة، تحتاج المكونات (Components) المختلفة إلى مشاركة البيانات فيما بينها، حتى لو لم تكن مرتبطة مباشرة (مكون أب/ابن). تُعد الخدمات (Services) و`BehaviorSubject` من الأدوات الفعالة لإدارة هذه الحالة المركزية وجعل البيانات تفاعلية (reactive)، مما يضمن تزامن تحديثات البيانات عبر جميع أجزاء التطبيق.

#### المفهوم الأساسي: الخدمة كمخزن مركزي

نستخدم الخدمة كمخزن مركزي للبيانات المشتركة. `BehaviorSubject` هو نوع خاص من المراقب (Observable) يحتفظ بآخر قيمة تم إرسالها. عندما يقوم مكون بتحديث القيمة في `BehaviorSubject`، فإن جميع المكونات الأخرى المشتركة في هذه القيمة تتلقى التحديث تلقائيًا.

#### مثال: خدمة مشاركة البيانات (DataService)

هذا المثال يوضح كيفية إنشاء خدمة تحتوي على `BehaviorSubject` لمشاركة رسالة بين المكونات:

```
// data.service.ts
import { Injectable } from '@angular/core';
import { BehaviorSubject } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
```



```
// القيمة الافتراضية BehaviorSubject إنشاء 1.
private messageSource = new BehaviorSubject<string>('الرسالة الافتراضية');

// 2. observable توفير عام للمكونات للاشتراك فيه
currentMessage = this.messageSource.asObservable();

constructor() { }

// 3. BehaviorSubject طريقة لتحديث قيمة
changeMessage(message: string) {
    this.messageSource.next(message);
}
}
```

## شرح الكود

يحتوي الكود على ثلاث خطوات رئيسية:

- **messageSource**: هو المتغير الرئيسي من نوع `BehaviorSubject`، ويبدأ بقيمة افتراضية ('الرسالة الافتراضية'). هذا المتغير هو الذي سيحتفظ بحالة البيانات.
- **currentMessage**: هو نسخة عامة (public) من `messageSource` كـ `Observable`. المكونات تقوم بالاشتراك في هذا المتغير (`subscribe`) لتلقي التحديثات دون التفاعل مباشرة مع `BehaviorSubject`.
- **changeMessage(message)**: هذه الطريقة تُستخدم لتحديث القيمة. عند استدعائها، فإن `BehaviorSubject` يقوم بإصدار القيمة الجديدة لجميع المكونات المشتركة.
- الخدمات توفر طريقة مركزية لتخزين البيانات المشتركة بين المكونات.
- `BehaviorSubject` يجعل البيانات تفاعلية (Reactive) بحيث تتلقى المكونات تحديثات فورية عند تغيير القيمة.
- هذا النمط يقلل من تعقيد تمرير البيانات بين المكونات غير المرتبطة مباشرة ويحسن تنظيم الكود.

# إدارة الحالة الأساسية (Basic State Management)

## — استخدام التخزين المحلي (LocalStorage) للحالة البسيطة

html

### إدارة الحالة الأساسية: استخدام LocalStorage

في تطبيقات Angular، نحتاج غالبًا إلى الاحتفاظ ببعض البيانات حتى بعد قيام المستخدم بتحديث الصفحة. تُعد آلية LocalStorage في المتصفح حلاً بسيطًا وفعالاً لإدارة الحالة البسيطة (مثل تفضيلات المستخدم أو إعدادات المظهر) دون الحاجة إلى قاعدة بيانات خلفية.

#### مفهوم LocalStorage

Local Storage (التخزين المحلي) هو واجهة برمجة تطبيقات (API) يوفرها المتصفح لتخزين البيانات محليًا على جهاز المستخدم. يتم تخزين البيانات على شكل أزواج من المفاتيح والقيم (key-value pairs). على عكس تخزين الجلسة (session storage)، تبقى البيانات المخزنة في LocalStorage متاحة حتى بعد إغلاق المتصفح أو إعادة تشغيل الجهاز.

#### مثال عملي: خدمة تخزين البيانات

لإنشاء نظام لإدارة الحالة باستخدام LocalStorage، نقوم عادةً بإنشاء خدمة (Service) لإخفاء منطق التخزين عن المكونات.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class StorageService {

  constructor() { }
```

```
// حفظ البيانات في LocalStorage
saveData(key: string, value: any): void {
  localStorage.setItem(key, JSON.stringify(value));
}

// استرجاع البيانات من LocalStorage
getData(key: string): any {
  const data = localStorage.getItem(key);
  return data ? JSON.parse(data) : null;
}
}
```

## شرح الكود

يقوم هذا الكود بإنشاء خدمة بسيطة (StorageService) تحتوي على وظيفتين أساسيتين: `saveData` و `getData`. عند حفظ البيانات، نستخدم `JSON.stringify` لتحويل كائن JavaScript إلى سلسلة نصية (string) لأن LocalStorage لا يخزن سوى السلاسل النصية. وعند استرجاع البيانات، نستخدم `JSON.parse` لإعادة تحويل السلسلة النصية إلى كائن JavaScript يمكن التعامل معه داخل التطبيق.

## ملخص الفكرة

- **التخزين الدائم:** LocalStorage يحافظ على البيانات بشكل دائم حتى بعد إغلاق المتصفح.
- **البيانات النصية:** يجب تحويل البيانات إلى سلاسل نصية (`JSON.stringify`) قبل التخزين وإعادة إنتاجها (`JSON.parse`) عند الاسترجاع.
- **الحالة البسيطة:** مناسبة لتخزين إعدادات المستخدم أو حالة المكونات غير المعقدة.